



# **MIT Art, Design and Technology University MIT School of Computing, Pune**

**Department of Information Technology**

## **Lab File**

**Practical – Full Stack Development**

**Class - L.Y. (SEM-I), SMAD**

**Name of the Practical Coordinator**

**Prof. Rohini Bhosale**

**Submitted By-  
Sujit Bhagwat**

**A.Y. 2025 – 2026 (SEM-VII)**

## COURSE STRUCTURE

| <b>Full Stack Development</b><br><b>SEMESTER – VII</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |              |                        |           |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------|------------------------|-----------|
| <b>Course Code:</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |              | <b>Course Credits:</b> | <b>04</b> |
| <b>Teaching Hours / Week (L:T:P):</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | <b>2:0:4</b> | <b>CA Marks:</b>       | <b>40</b> |
| <b>Total Number of Teaching Hours:</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | <b>–</b>     | <b>END-SEM Marks:</b>  | <b>60</b> |
| <b>Course Pre-requisites:</b> Web Technology                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |              |                        |           |
| <b>Course Description:</b><br>a. This course offers a comprehensive foundation in full stack web development using Node.js, Express.js, EJS, and MySQL.<br>b. Students will learn to build dynamic and responsive web applications by integrating front-end and back-end technologies.<br>c. The course emphasizes hands-on project-based learning and real-world application development.<br>d. Learners will design RESTful APIs and manage databases for seamless client-server interaction.<br>e. Through practical exercises, students will gain industry-ready skills for scalable web application development.                                                                                                                                                  |              |                        |           |
| <b>Course Learning Objectives:</b> This course will enable the students to: <ol style="list-style-type: none"> <li>1. Introduce students to server-side development using Node.js and its ecosystem.</li> <li>2. Equip learners with the ability to develop scalable and efficient web servers using Express.js.</li> <li>3. Enable the use of EJS for dynamic web page rendering and data binding.</li> <li>4. Provide hands-on experience in integrating relational databases like MySQL with Node.js.</li> <li>5. Facilitate the development of full stack applications through project-based learning using REST APIs and client-server integration.</li> </ol>                                                                                                    |              |                        |           |
| <b>Course Outcome:</b> <ol style="list-style-type: none"> <li>1. CO1: Build responsive front-end user interfaces using Bootstrap layout and component system.</li> <li>2. CO2: Analyze the core principles, architecture, and modular structure of Node.js for robust server-side application development.</li> <li>3. CO3: Construct and deploy scalable web servers using Node.js and Express.js to efficiently manage HTTP protocols and client-server communication.</li> <li>4. CO4: Design and synthesize dynamic, data-driven user interfaces using EJS templating integrated with backend services.</li> <li>5. CO5: Implement and manage CRUD operations by interfacing Node.js with MySQL and engineer RESTful APIs for structured data exchange.</li> </ol> |              |                        |           |

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |                                                           |                 |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|-----------------|
| <b>UNIT – I</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | <b>Responsive Interfaces with Bootstrap Components</b>    | <b>14 Hours</b> |
| <b>Main Topic-1: Introduction to Media Queries, Responsive Web Design Using Media Queries, Syntax, Uses of Media Queries, Benefits of Responsive Design, Elements of a Responsive Design, Working of Responsive Design</b><br><b>Main Topic-2: Introduction to Bootstrap: Bootstrap Concept, Uses of Bootstrap, Bootstrap library source, Bootstrap Basics- container, jumbotron, page header, buttons, tables;</b><br><b>Main Topic-3: Bootstrap Grid System, Bootstrap Images, Bootstrap Forms, Bootstrap Flexbox- horizontal direction, vertical direction.</b> |                                                           |                 |
| <b>Pedagogy</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | <b>ICT Teaching / Power Point Presentation and Videos</b> |                 |
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | <b>Self-study / Do it yourself</b>                        |                 |
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | <b>Experiential Learning Topics:</b>                      |                 |
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | <b>Case Study / PBL - Project Based Learning</b>          |                 |
| <b>UNIT – II</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | <b>Backend Development Fundamentals with Node.js</b>      | <b>10 Hours</b> |
| <b>Main Topic-1: Introduction to Node.js: What is Node.js? Why use it? Capabilities and file structure</b><br><b>Main Topic-2: Setting up Node.js Development Environment – Installing Node.js and npm, NPM and project creation, Node.js modules, Creating a basic web server, Node.js console</b>                                                                                                                                                                                                                                                                |                                                           |                 |
| <b>Pedagogy</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | <b>ICT Teaching / Power Point Presentation and Videos</b> |                 |
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | <b>Self-study / Do it yourself</b>                        |                 |
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | <b>Experiential Learning Topics:</b>                      |                 |
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | <b>Case Study / PBL - Project Based Learning</b>          |                 |
| <b>UNIT – III</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | <b>Building Web Servers using Express.js</b>              | <b>12 Hours</b> |
| <b>Main Topic-1: Introduction to Express.js – Features, benefits, use cases</b><br><b>Main Topic-2: Express Server Setup – Installing Express.js, project setup, configuration, building Express - based web server</b>                                                                                                                                                                                                                                                                                                                                            |                                                           |                 |
| <b>Pedagogy</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | <b>ICT Teaching / Power Point Presentation and Videos</b> |                 |
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | <b>Self-study / Do it yourself</b>                        |                 |
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | <b>Experiential Learning Topics:</b>                      |                 |
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | <b>Case Study / PBL - Project Based Learning</b>          |                 |

|                                                                                                                                                                                                                                                                                                                                           |                                                           |                 |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|-----------------|
| <b>UNIT – IV</b>                                                                                                                                                                                                                                                                                                                          | <b>Templating and Dynamic Views with EJS</b>              | <b>12 Hours</b> |
| <b>Main Topic-1: Integrating EJS Templates, Creating EJS partials and views, Passing data to views, rendering variables, Looping through data in templates</b><br><b>Main Topic-2: Building dynamic front-ends using EJS with Express</b>                                                                                                 |                                                           |                 |
| <b>Pedagogy</b>                                                                                                                                                                                                                                                                                                                           | <b>ICT Teaching / Power Point Presentation and Videos</b> |                 |
|                                                                                                                                                                                                                                                                                                                                           | <b>Self-study / Do it yourself</b>                        |                 |
|                                                                                                                                                                                                                                                                                                                                           | <b>Experiential Learning Topics:</b>                      |                 |
|                                                                                                                                                                                                                                                                                                                                           | <b>Case Study / PBL - Project Based Learning</b>          |                 |
| <b>UNIT – V</b>                                                                                                                                                                                                                                                                                                                           | <b>Database Integration and REST API Development</b>      | <b>12 Hours</b> |
| <b>Main Topic-1: Connecting Node.js with MySQL, Installing MySQL, setting up the connection, Performing CRUD operations using Node.js and MySQL</b><br><b>Main Topic-2: Building RESTful APIs using Express.js, HTTP methods (GET, POST, PUT, DELETE), Handling requests and responses using Express, Using body-parser for form data</b> |                                                           |                 |
| <b>Pedagogy</b>                                                                                                                                                                                                                                                                                                                           | <b>ICT Teaching / Power Point Presentation and Videos</b> |                 |
|                                                                                                                                                                                                                                                                                                                                           | <b>Self-study / Do it yourself</b>                        |                 |
|                                                                                                                                                                                                                                                                                                                                           | <b>Experiential Learning Topics:</b>                      |                 |
|                                                                                                                                                                                                                                                                                                                                           | <b>Case Study / PBL - Project Based Learning</b>          |                 |

### Text Books:

1. Andrew Mead, [Learning Node.js Development: Learn the fundamentals of Node.js, and deploy and test Node.js applications on the web], Packt Publishing, ISBN-10 : 1788395549, ISBN-13 : 978-1788395540
2. Ethan Brown, [Web Development with Node and Express, 2e: Leveraging the JavaScript Stack], O'Reilly; 2nd edition, ISBN-10 : 1492053511, ISBN-13 : 978-1492053514

### Reference Books:

1. Mario Casciaro and Luciano Mammino, [Node.js Design Patterns], Packt Publishing, ISBN: 978-1839214110
2. Russell Dyer, [Learning MySQL and MariaDB], O'Reilly Media, ISBN: 978-1449362904
3. Leonard Richardson and Mike Amundsen, [RESTful Web APIs], O'Reilly Media, ISBN: 978-1449358068
4. Ben Frain, Responsive Web Design with HTML5 and CSS (3e), Packt Publishing, ISBN: 978-1839211560

### URLs (Optional) - List of Online Courses

1. [Full-Stack Web Development with React Specialization – Coursera \(by The Hong Kong University of Science and Technology\)](#)



Covers: Node.js, Express.js, MongoDB, React, REST APIs  
 Platform: Coursera  
 Level: Intermediate – Project-based

2. **The Complete Node.js Developer Course (3rd Edition) – Udemy (by Andrew Mead & Rob Percival)**

Covers: Node.js, Express, MongoDB, REST APIs, Authentication  
 Platform: Udemy  
 Level: Beginner to Advanced

3. **[Web Development with Node.js and Express – Coursera \(by University of London\)](#)**

Covers: Node.js, Express.js, REST APIs  
 Platform: Coursera  
 Level: Intermediate

4. **MySQL for Data Analytics and Business Intelligence – Udemy**

Covers: MySQL Basics to Advanced (joins, CRUD, queries)  
 Platform: Udemy  
 Level: Beginner to Intermediate

5. **Full Stack Open 2024 – University of Helsinki (Free)**

Covers: Modern web app development with React, Node.js, REST APIs, MongoDB  
 Platform: Independent (Free)  
 Level: Advanced

**List of Assignments:**

| Sr.<br>No. | Assignment Title                                                                                                                                                                                                                                                               |
|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1.         | <b>Create a Static Homepage Layout Using Bootstrap</b> <p>1.1 Design a responsive homepage using Bootstrap components like container, jumbotron, headers, and buttons.</p> <p>1.2 Use the grid system to create a structured and adaptive layout suitable for any web app.</p> |
| 2.         | <b>Design a Grid-Based Information Page</b> <p>2.1 Develop a multi-column content layout using Bootstrap Grid.</p> <p>2.2 Display items such as cards, product previews, or blog entries with consistent alignment and spacing.</p>                                            |
| 3.         | <b>Set Up a Node.js Project with Express Server</b> <p>3.1 Initialize a new Node.js project using npm init.</p> <p>3.2 Install Express.js and set up a basic web server with routes like Home (/), About (/about), and Contact (/contact).</p>                                 |
| 4.         | <b>Implement Modular Routing and Middleware Logging</b> <p>4.1 Organize server logic using separate route modules (e.g., userRoutes, productRoutes).</p> <p>4.2 Add middleware to log request URLs, methods, and timestamps for all incoming traffic.</p>                      |

## 5. Create a Data Submission Form and Handle POST Requests

5.1 Build an HTML form (e.g., for adding items or submitting feedback) and handle its POST request using Express.

5.2 Display the submitted data dynamically on a confirmation view.

## 6. Serve Dynamic Content Using Server-Side Rendering

6.1 Render a list of items (e.g., products, posts, services) dynamically using EJS with hardcoded or temporary in-memory data.

6.2 Route should accept data from the server and render it in the view using loops.

## 7. Create Dynamic Detail Pages Using URL Parameters

7.1 Use route parameters to render individual item details dynamically via EJS.

7.2 Display properties like title, description, and date using passed route data.

## 8. Build a Reusable Layout Using EJS Partial

8.1 Create common EJS partials (header, footer, navbar) and include them across views.

8.2 Maintain consistency across all pages by using a shared layout template.

## 9. Connect Node.js to MySQL and Perform Basic CRUD

9.1 Create MySQL tables and connect to them using a MySQL Node.js package.

9.2 Implement basic CRUD operations from Express routes to the database.

## 10. Develop RESTful APIs for Data Access and Manipulation

10.1 Build a set of RESTful APIs using Express.js for performing operations.

10.2 Test the endpoints using Postman and ensure they return structured JSON responses.

| ASSESSMENT AND EVALUATION PATTERN                                                                        |                            |                               |
|----------------------------------------------------------------------------------------------------------|----------------------------|-------------------------------|
| WEIGHTAGE                                                                                                | Continuous Assessment (CA) | End Semester Assessment (ESA) |
| CA - Assignments (PBL / Case Study / Presentation / Seminar / Group Discussions / Quiz / Test            |                            |                               |
| ESA - Bloom's Taxonomy Levels: Remembering, Understanding, Applying, Analyzing, Evaluating, and Creating |                            |                               |
| Experiential Learning - Internship (Summer / Winter) / Industry Visit / Site Visit / Field Trips         |                            |                               |
| Project Based Learning – Mini (Minor) Project / Major Project                                            |                            |                               |

| Course Articulation Matrix (CO-PO Mapping) [Subject Title – Subject Code] |      |      |      |      |      |      |      |      |      |       |       |       |       |       |
|---------------------------------------------------------------------------|------|------|------|------|------|------|------|------|------|-------|-------|-------|-------|-------|
| COs                                                                       | PO-1 | PO-2 | PO-3 | PO-4 | PO-5 | PO-6 | PO-7 | PO-8 | PO-9 | PO-10 | PO-11 | PO-12 | PSO-1 | PSO-2 |
| CO-1                                                                      | 3    | 3    | 2    | 3    | 3    |      |      |      |      | 3     |       | 3     | 2     |       |
| CO-2                                                                      | 3    | 2    |      |      | 2    |      |      |      |      | 3     |       | 3     | 2     |       |
| CO-3                                                                      | 3    | 3    | 2    | 3    | 3    |      |      |      |      | 2     |       | 3     | 2     |       |
| CO-4                                                                      |      |      |      |      | 3    |      |      |      | 2    | 3     |       | 2     |       | 3     |
| CO-5                                                                      |      |      |      |      | 3    |      |      |      | 2    | 3     |       | 2     |       | 3     |



## Experiment No-01

### Problem Statement -

1. Create a Static Homepage Layout Using Bootstrap

1.1 Design a responsive homepage using Bootstrap components like container, jumbotron, headers, and buttons.

1.2 Use the grid system to create a structured and adaptive layout suitable for any web app

### Theory-

#### Introduction to Bootstrap

Bootstrap is an open-source, front-end framework developed by Twitter that allows developers to design responsive and mobile-first websites quickly. It includes a comprehensive collection of HTML, CSS, and JavaScript components that make it easier to build modern user interfaces.

Bootstrap simplifies layout creation through its **12-column grid system**, responsive utilities, and pre-styled UI elements like **buttons, forms, navigation bars, modals, and cards**. It ensures consistency across browsers and devices, drastically reducing development time.

#### 1. Responsive Web Design

Responsive design ensures that a webpage automatically adjusts its layout and elements to fit different screen sizes — desktops, tablets, and mobile devices. Bootstrap achieves responsiveness using fluid grids, flexible images, and media queries.

Bootstrap uses four key breakpoints:

- xs (extra small devices like phones)
- sm (small devices like tablets)
- md (medium devices like laptops)
- lg and xl (large and extra-large devices like desktops)

By combining these breakpoints with grid classes, developers can design adaptive layouts efficiently.

#### 2. Bootstrap Container

The **container** is the base element that wraps the entire content and provides proper alignment and padding.

Bootstrap offers:

- .container – a fixed-width container that adjusts with breakpoints.
- .container-fluid – a full-width container spanning the entire screen width.

Example:

```
<div class="container">
```

```
<h1>Welcome to Our Homepage</h1>
```

```
</div>
```

### 3. Grid System

The **grid system** divides the page into 12 columns. Developers can combine columns using classes like `.col-md-4` or `.col-sm-6` to control how many columns each section takes up on different devices.

Example:

```
<div class="row">
```

```
<div class="col-md-4">Section 1</div>
```

```
<div class="col-md-4">Section 2</div>
```

```
<div class="col-md-4">Section 3</div>
```

```
</div>
```

This ensures a flexible and responsive structure

### 4. Jumbotron (Hero Section)

A **jumbotron** (or “hero section” in newer versions) is used to highlight key information like a welcome message or tagline. It draws user attention and often includes a call-to-action button.

Example:

```
<div class="jumbotron text-center bg-light">
```

```
<h1>Welcome to Our Web App</h1>
```

```
<p>Building responsive designs with ease using Bootstrap</p>
```

```
<a href="#" class="btn btn-primary">Get Started</a>
```

```
</div>
```

### 5. Headers and Buttons

Headers (`<h1>` to `<h6>`) establish content hierarchy, while **Bootstrap buttons** (`.btn`, `.btn-primary`, `.btn-outline-success`, etc.) enhance interactivity with predefined styles.

These elements combine to create an attractive and user-friendly homepage, maintaining design consistency across all devices.

## Code-

### Index.html-

```
<!DOCTYPE html>

<html lang="en">

<head>

<meta charset="UTF-8" />

<meta name="viewport" content="width=device-width, initial-scale=1.0" />

<title><%= pageTitle %></title>

<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet" />

<link href="https://unpkg.com/aos@2.3.1/dist/aos.css" rel="stylesheet">

<link href="https://fonts.googleapis.com/css2?family=Poppins:wght@400;600;700&display=swap"
rel="stylesheet" />

<style>

  body {

    font-family: 'Poppins', sans-serif;

    background-color: #0e0e0e;

    color: #f8f9fa;

    overflow-x: hidden;

    scroll-behavior: smooth;

  }

  .navbar {

    background: rgba(20, 20, 20, 0.95);

    backdrop-filter: blur(10px);

    box-shadow: 0 4px 25px rgba(0, 0, 0, 0.5);
```

```
    transition: background 0.4s ease;

}

.navbar-brand {

    font-size: 1.6rem;

    color: #00aaff !important;

    font-weight: 700;

}

.nav-link {

    color: #f8f9fa !important;

    margin: 0 8px;

    font-weight: 500;

    transition: color 0.3s ease;

}

.nav-link:hover {

    color: #00aaff !important;

}

header {

    background: linear-gradient(180deg, #111, #222);

    padding: 6rem 1rem;

    text-align: center;

    color: #fff;

}

header h1 {

    font-weight: 700;

    font-size: 3rem;
```

```
text-shadow: 0 3px 10px rgba(0, 170, 255, 0.4);

}

header p {

font-size: 1.2rem;

margin-bottom: 1.5rem;

opacity: 0.9;

}

header .btn {

background-color: #00aaff;

border: none;

font-weight: 600;

transition: transform 0.3s ease, background 0.3s ease;

}

header .btn:hover {

transform: scale(1.1);

background-color: #007acc;

}

section h2 {

color: #00aaff;

font-weight: 700;

margin-bottom: 2rem;

text-align: center;

}

.game-card {

background: #1b1b1b;
```



```
border: none;

border-radius: 14px;

transition: transform 0.4s ease, box-shadow 0.4s ease;

overflow: hidden;
}

.game-card:hover {

transform: translateY(-10px);

box-shadow: 0 10px 35px rgba(0, 170, 255, 0.4);

}

.game-card img {

border-radius: 14px 14px 0 0;

}

.news-card {

background: #1a1a1a;

border-radius: 12px;

padding: 1.5rem;

height: 100%;

transition: transform 0.4s ease, background 0.3s ease;

}

.news-card:hover {

transform: translateY(-8px);

background: #232323;

}

.news-card h4 {

color: #00aaff;
```

```
font-weight: 600;

margin-bottom: 0.6rem;

}
```

```
.stats {

background: linear-gradient(90deg, #0e0e0e, #1c1c1c);

padding: 4rem 1rem;

border-top: 1px solid #222;

border-bottom: 1px solid #222;

}
```

```
.stats h3 {

font-size: 2.4rem;

color: #00aaff;

}
```

```
.stats p {

color: #bbb;

font-size: 1rem;

}
```

```
footer {

background: #1b1b1b;

color: #bbb;

padding: 1.5rem 0;

text-align: center;

font-size: 0.9rem;

}
```

```
[data-aos] {
```

```

    transition-property: transform, opacity;

    transition-duration: 1s;
}

@media (max-width: 992px) {

    header h1 { font-size: 2.2rem; }

    header p { font-size: 1rem; }

}

@media (max-width: 576px) {

    header { padding: 3rem 1rem; }

    header h1 { font-size: 1.8rem; }

}

</style>

</head>

<body>

<nav class="navbar navbar-expand-lg navbar-dark fixed-top">

    <div class="container">

        <a class="navbar-brand" href="/"> 🎮 GameZone</a>

        <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-
target="#navbarNav">

            <span class="navbar-toggler-icon"></span>

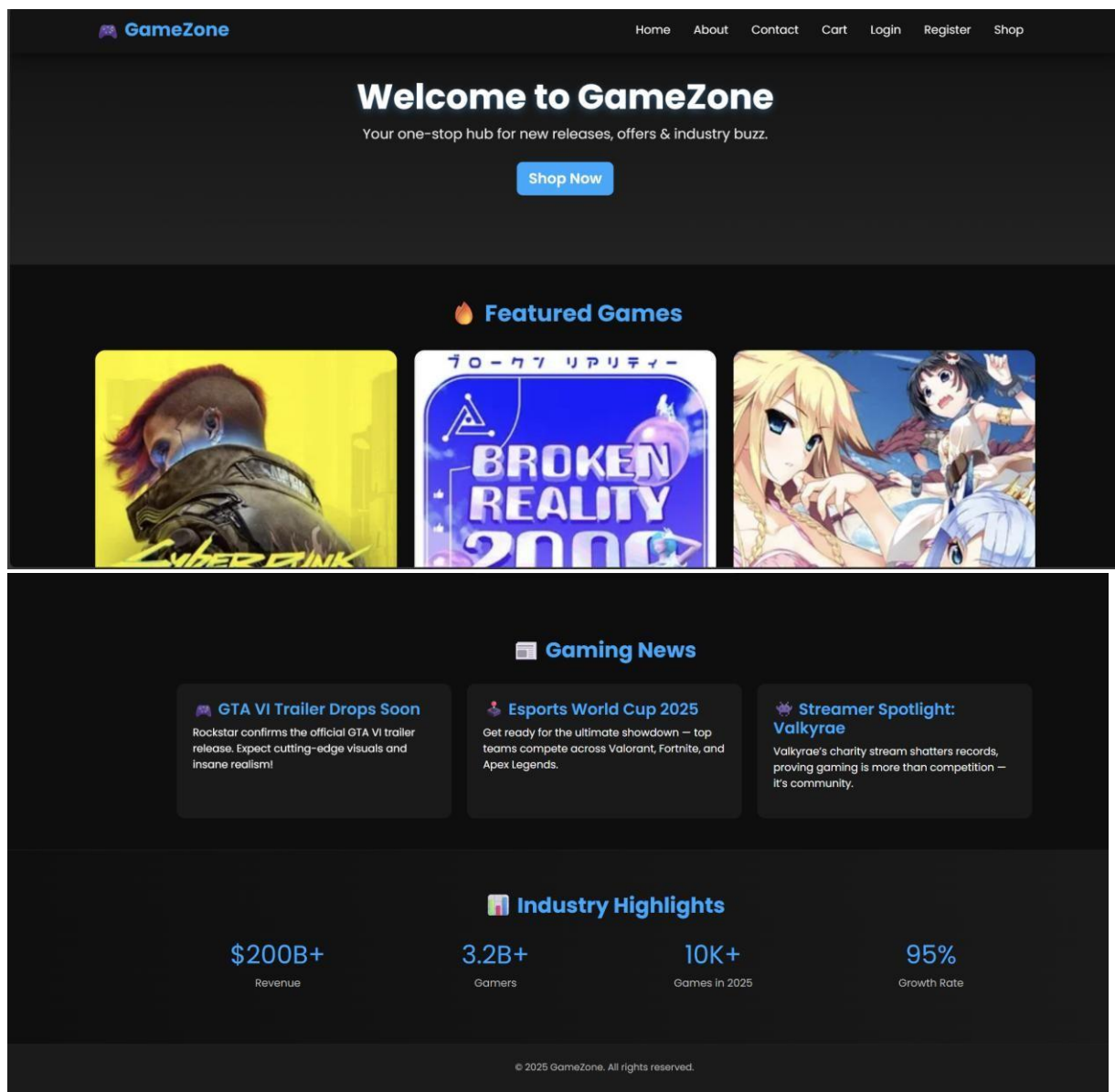
        </button>

        <div class="collapse navbar-collapse justify-content-end" id="navbarNav">

            <ul class="navbar-na

```

Output-



## Conclusion-

By designing a static homepage layout using Bootstrap's responsive features and grid system, developers can achieve a visually consistent and device-friendly interface. The implementation reduces design complexity, enhances user experience, and maintains a professional and modern look across all screen resolutions—making Bootstrap an ideal choice for building adaptive web application homepages.

## Experiment No.2

### Problem Statement-

The objective is to design a grid-based information page using Bootstrap that efficiently organizes and displays multiple content elements such as cards, product previews, or blog entries. The page must utilize the Bootstrap Grid system to ensure consistent alignment, equal spacing, and responsiveness across all screen sizes, offering a visually appealing and structured presentation suitable for modern web applications.

### Theory-

Bootstrap, developed by Twitter, is one of the most popular **front-end frameworks** for building responsive, mobile-first web applications. At its core lies the **Grid System**, a flexible layout model based on a **12-column structure**. This system allows developers to divide the webpage into rows and columns, enabling precise control over content alignment and distribution. Each column is represented using classes like `.col-sm-6`, `.col-md-4`, `.col-lg-3`, etc., which automatically adjust the number of visible columns depending on the screen size.

For example, using `.col-md-4` divides a row into three equal columns on medium-sized screens, while `.col-sm-6` creates two columns on smaller screens. This adaptability ensures that the webpage remains **user-friendly** and visually consistent across all devices.

To enhance the layout, Bootstrap offers a range of **predefined components**—most notably **cards**—that encapsulate information in small, neatly designed containers. A card typically includes an image, title, description, and a call-to-action button. Cards are ideal for representing products, blog posts, user profiles, or feature highlights in a structured grid layout. The use of Bootstrap's **utility classes** like `p-3` (padding), `m-2` (margin), and `text-center` (alignment) further enhances the design by ensuring proper spacing, readability, and alignment.

In addition, the grid system allows for **nesting**, where columns can contain rows within them to create more complex layouts. This makes it easier to organize diverse types of content without breaking the design's flow. By combining the grid with components like **containers**, **rows**, and **columns**, developers can build layouts that are not only visually appealing but also semantically structured and accessible.

Moreover, the use of **responsive design principles** ensures that the content automatically adjusts to any device's screen width, eliminating the need for separate designs for different platforms. This approach improves performance, accessibility, and user satisfaction.

### Code-

About.html-

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
<title>About Us - GameZone</title>
<link
  href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
  rel="stylesheet"
/>
<style>
  body {
    font-family: 'Poppins', sans-serif;
    background: linear-gradient(135deg, #1b1b1b, #2d2d2d);
    color: #fff;
    overflow-x: hidden;
  }
  .navbar {
    box-shadow: 0 4px 10px rgba(0,0,0,0.5);
    transition: background 0.4s ease;
  }
  .navbar:hover {
    background: rgba(0,0,0,0.95);
  }
  .nav-link {
    position: relative;
    font-weight: 500;
    transition: all 0.3s ease;
  }
  .nav-link:hover {
    color: #0d6efd !important;
  }
  .nav-link::after {
    content: "";
    position: absolute;
    width: 0%;
    height: 2px;
    background: #0d6efd;
    left: 0;
    bottom: 0;
    transition: width 0.3s ease;
  }
  .nav-link:hover::after {
    width: 100%;
  }
}
```

```
header {
  background: linear-gradient(135deg, #0d6efd, #6610f2);
  color: #fff;
  text-align: center;
  padding: 6rem 1rem;
  animation: fadeInDown 1s ease-in-out;
}

header h1 {
  font-size: 3rem;
  font-weight: 700;
  animation: slideIn 1s ease-in-out;
}

header p {
  font-size: 1.2rem;
  opacity: 0.9;
  animation: fadeIn 1.5s ease-in-out;
}

section.container {
  animation: fadeUp 1.5s ease-in-out;
}

.img-fluid {
  border-radius: 15px;
  box-shadow: 0 0 25px rgba(0,0,0,0.4);
  transition: transform 0.3s ease, box-shadow 0.3s ease;
}

.img-fluid:hover {
  transform: scale(1.03);
  box-shadow: 0 0 35px rgba(13,110,253,0.4);
}

.btn-primary {
  background: linear-gradient(135deg, #0d6efd, #6610f2);
  border: none;
  transition: transform 0.3s ease, background 0.3s ease;
  font-weight: 600;
}

.btn-primary:hover {
  background: linear-gradient(135deg, #6610f2, #0d6efd);
  transform: scale(1.05);
}

.bg-light {
  background: rgba(255, 255, 255, 0.05) !important;
```

```

    backdrop-filter: blur(10px);
    color: #fff;
  }
  .border {
    border: 1px solid rgba(255, 255, 255, 0.2) !important;
  }
  .shadow-sm {
    box-shadow: 0 4px 20px rgba(0,0,0,0.3) !important;
    transition: transform 0.3s ease, box-shadow 0.3s ease;
  }
  .shadow-sm:hover {
    transform: translateY(-5px);
    box-shadow: 0 6px 25px rgba(13,110,253,0.4) !important;
  }

  footer {
    background: #000;
    color: #ccc;
    font-size: 0.9rem;
    letter-spacing: 0.5px;
    animation: fadeIn 2s ease-in-out;
  }

  @keyframes fadeIn {
    from { opacity: 0; }
    to { opacity: 1; }
  }

  @keyframes fadeInDown {
    from {
      opacity: 0;
      transform: translateY(-40px);
    }
    to {
      opacity: 1;
      transform: translateY(0);
    }
  }

  @keyframes fadeUp {
    from {
      opacity: 0;
      transform: translateY(50px);
    }
    to {

```



```

    opacity: 1;
    transform: translateY(0);
  }
}
@keyframes slideIn {
  from {
    opacity: 0;
    transform: translateX(-50px);
  }
  to {
    opacity: 1;
    transform: translateX(0);
  }
}
@media (max-width: 768px) {
  header h1 {
    font-size: 2rem;
  }
  .img-fluid {
    margin-bottom: 1.5rem;
  }
}
</style>
</head>
<body>
<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
  <div class="container">
    <a class="navbar-brand fw-bold" href="/"> 🎮 GameZone</a>
    <button
      class="navbar-toggler"
      type="button"
      data-bs-toggle="collapse"
      data-bs-target="#navbarNav"
      title="Toggle navigation"
      aria-label="Toggle navigation"
    >
      <span class="navbar-toggler-icon"></span>
    </button>
  <div class="collapse navbar-collapse" id="navbarNav">
    <ul class="navbar-nav ms-auto">
      <li class="nav-item"><a class="nav-link" href="/">Home</a></li>
      <li class="nav-item"><a class="nav-link active" href="/about">About</a></li>
    </ul>
  </div>
</div>
</nav>

```

```

    <li class="nav-item"><a class="nav-link" href="/contact">Contact</a></li>
    <li class="nav-item"><a class="nav-link" href="/cart">Cart</a></li>
    <li class="nav-item"><a class="nav-link" href="/login">Login</a></li>
    <li class="nav-item"><a class="nav-link" href="/register">Register</a></li>
  </ul>
</div>
</div>
</nav>

```

```

<header>
  <div class="container">
    <h1 class="fw-bold">About GameZone</h1>
    <p class="lead">Your Ultimate Destination for Gaming Adventures</p>
  </div>
</header>

```

```

<section class="container py-5">
  <div class="row align-items-center g-4">
    <div class="col-md-6 text-center">
      
    </div>

```

```

    <div class="col-md-6">
      <h2 class="fw-bold mb-3">Who We Are</h2>
      <p>
        At <strong>GameZone</strong>, we are passionate gamers who believe in
        bringing the best gaming experiences to everyone. Our store features a
        wide range of the latest releases, timeless classics, and exclusive
        titles — all in one place.

```

```

      </p>
      <p>
        Whether you are into action, adventure, sports, or strategy games, we
        have something for everyone. We're committed to offering affordable
        prices, secure shopping, and the best customer support.
      </p>
      <a href="/contact" class="btn btn-primary">Get in Touch</a>

```

```

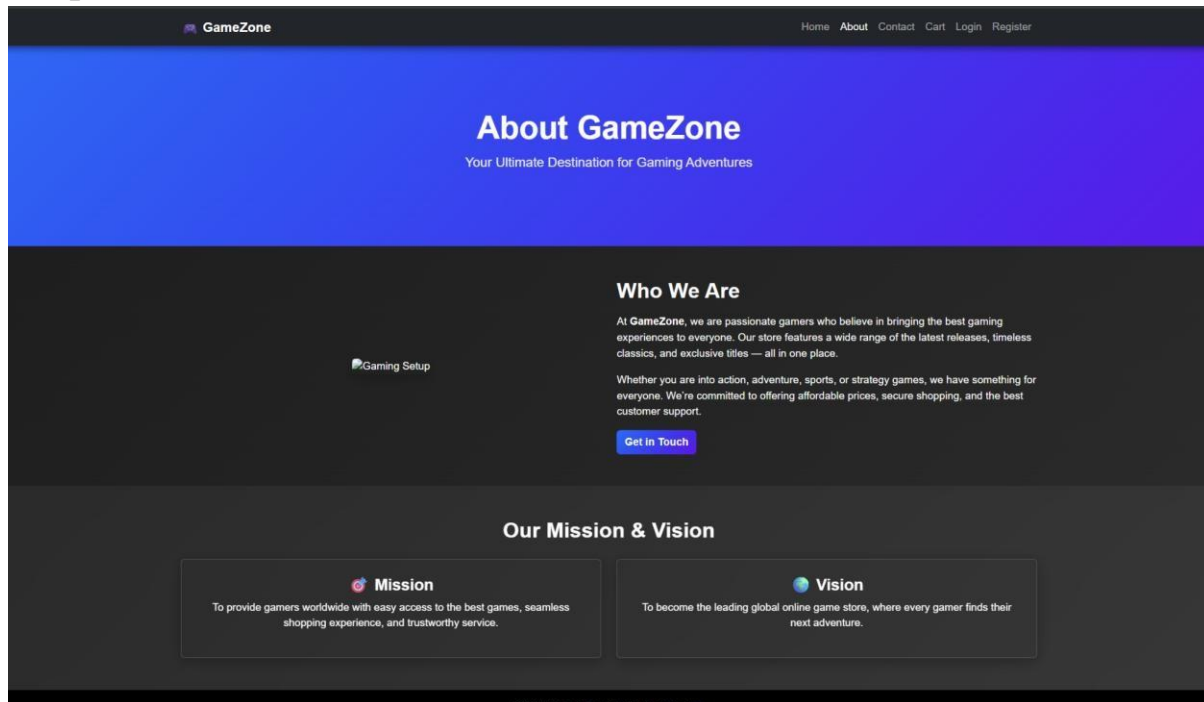
    </div>
  </div>
</section>

<section class="bg-light py-5">
  <div class="container text-center">
    <h2 class="fw-bold mb-4">Our Mission & Vision</h2>
    <div class="row g-4">
      <div class="col-md-6">
        <div class="p-4 border rounded shadow-sm h-100">
          <h4 class="fw-bold">🎮 Mission</h4>
          <p>
            To provide gamers worldwide with easy access to the best games,
            seamless shopping experience, and trustworthy service.
          </p>
        </div>
      </div>
      <div class="col-md-6">
        <div class="p-4 border rounded shadow-sm h-100">
          <h4 class="fw-bold">🌐 Vision</h4>
          <p>
            To become the leading global online game store, where every gamer
            finds their next adventure.
          </p>
        </div>
      </div>
    </div>
  </div>
</section>

<footer class="py-3 text-center">
  <p class="mb-0">&copy; 2025 GameZone. All rights reserved 🎮</p>
</footer>
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
<script>
  document.addEventListener("DOMContentLoaded", () => {
    console.log("Welcome to GameZone — where every click starts a new adventure!");
  });
</script>
</body>
</html>

```

## Output-



## Cart.html-

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Cart - GameZone</title>

  <!-- Bootstrap CSS -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet" />
  <link href="https://cdnjs.cloudflare.com/ajax/libs/animate.css/4.1.1/animate.min.css" rel="stylesheet" />

  <style>
    body {
      background-color: #121212;
      color: #fff;
      font-family: "Poppins", sans-serif;
    }

    .navbar {
      box-shadow: 0 4px 8px rgba(0,0,0,0.4);
    }

    .navbar-brand {
```

```
font-size: 1.5rem;
font-weight: bold;
letter-spacing: 1px;
}

.nav-link {
  color: #ccc !important;
  margin: 0 10px;
  transition: 0.3s;
}

.nav-link:hover {
  color: #0dcaf0 !important;
  transform: scale(1.05);
}

h1 {
  text-align: center;
  margin-bottom: 2rem;
  font-weight: 700;
  letter-spacing: 1px;
  animation: fadeInDown 1s;
}

#cart-container {
  background: #f1f1f1;
  border-radius: 15px;
  padding: 2rem;
  box-shadow: 0 0 15px rgba(0,0,0,0.6);
  animation: fadeInUp 1.2s;
}

.cart-item {
  display: flex;
  align-items: center;
  justify-content: space-between;
  background: #2c2c2c;
  padding: 15px;
  margin-bottom: 10px;
  border-radius: 10px;
  transition: all 0.3s ease;
}
```

```
.cart-item:hover {  
  background: #343434;  
  transform: translateY(-3px);  
}  
  
.cart-item img {  
  width: 80px;  
  height: 80px;  
  border-radius: 10px;  
  margin-right: 15px;  
}  
  
.item-info {  
  flex: 1;  
}  
  
.total-section {  
  text-align: right;  
  margin-top: 1.5rem;  
}  
  
.btn-checkout {  
  background: linear-gradient(90deg, #0dcaf0, #6610f2);  
  border: none;  
  padding: 10px 25px;  
  border-radius: 30px;  
  color: #fff;  
  font-weight: bold;  
  transition: 0.3s;  
}  
  
.btn-checkout:hover {  
  transform: scale(1.05);  
  opacity: 0.9;  
}  
  
footer {  
  background: #0d0d0d;  
  margin-top: 3rem;  
  padding: 1rem;  
  color: #bbb;
```

```

    }
</style>
</head>
<body>
  <nav class="navbar navbar-expand-lg navbar-dark bg-dark py-3">
    <div class="container">
      <a class="navbar-brand" href="/">🎮 GameZone</a>
      <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navMenu">
        <span class="navbar-toggler-icon"></span>
      </button>
    <div class="collapse navbar-collapse justify-content-end" id="navMenu">
      <ul class="navbar-nav">
        <li class="nav-item"><a class="nav-link" href="/">Home</a></li>
        <li class="nav-item"><a class="nav-link" href="/about">About</a></li>
        <li class="nav-item"><a class="nav-link" href="/contact">Contact</a></li>
        <li class="nav-item"><a class="nav-link active text-info" href="/cart">Cart</a></li>
        <li class="nav-item"><a class="nav-link" href="/login">Login</a></li>
        <li class="nav-item"><a class="nav-link" href="/register">Register</a></li>
      </ul>
    </div>
  </div>
</nav>

<!-- Cart Section -->
<main class="container my-5">
  <div id="cart-container">
    <h1>Your Cart 🛒</h1>
    <div id="cart-items">
      <div class="cart-item">
        
        <div class="item-info">
          <h5>Counter-Strike 2</h5>
          <p class="mb-0 text-info">$19.99</p>
        </div>
        <button class="btn btn-sm btn-danger">Remove</button>
      </div>
      <div class="cart-item">
        
        <div class="item-info">
          <h5>GTA V</h5>
          <p class="mb-0 text-info">$29.99</p>
        </div>
      </div>
    </div>
  </div>
</main>

```

```

        <button class="btn btn-sm btn-danger">Remove</button>
    </div>
</div>

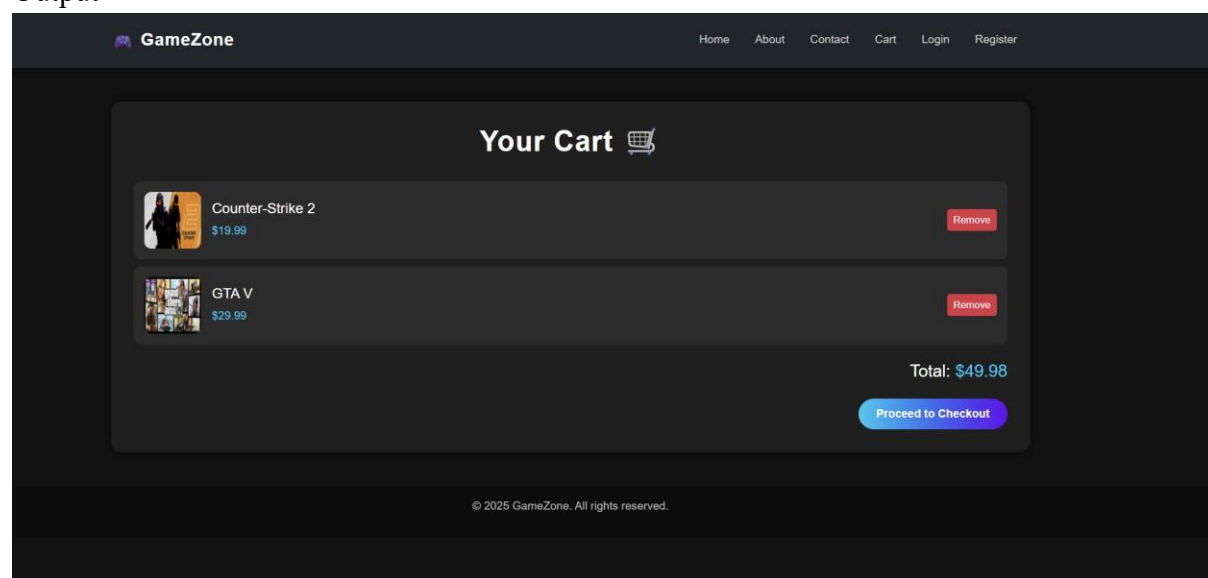
<div class="total-section">
    <h4>Total: <span class="text-info">$49.98</span></h4>
    <button class="btn-checkout mt-3">Proceed to Checkout</button>
</div>
</div>
</main>

<footer class="text-center">
    <p>&copy; 2025 GameZone. All rights reserved.</p>
</footer>

<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>

```

### Output-



### Conclusion -

A grid-based information page built using Bootstrap ensures a well-structured, responsive, and visually uniform layout that enhances content presentation. By combining the grid system with components like cards and spacing utilities, developers can deliver a professional and scalable design that maintains alignment, readability, and adaptability across various devices, ensuring an optimal user experience.



## Experiment No-03

### Problem Statement-

The objective of this project is to set up a Node.js-based web application for a Game Store using the Express.js framework. The goal is to initialize a new Node.js project with `npm init` and configure a basic web server that includes essential routes such as Home (/) for showcasing featured games, About (/about) for providing information about the Game Store and its mission, and Contact (/contact) for users to get in touch for support or inquiries. This setup will form the foundation of a dynamic Game Store web app, where future features like user authentication, game listings, and shopping cart functionality can be integrated seamlessly.

### Theory-

Node.js is a **runtime environment** built on Chrome's V8 JavaScript engine that allows developers to run JavaScript on the server side. It enables the development of fast, scalable, and efficient web applications. The core advantage of Node.js lies in its **non-blocking, event-driven architecture**, which allows multiple client requests to be handled concurrently without overloading the system.

To start a Node.js project, developers use the command `npm init`, which initializes the project and creates a `package.json` file containing metadata such as project name, version, scripts, and dependencies. This file serves as the blueprint of the project, managing installed libraries and ensuring version control.

**Express.js**, a minimal and flexible Node.js web framework, is widely used to build web servers and APIs efficiently. It simplifies the process of defining routes, handling HTTP requests, and serving static files. Once Express is installed using `npm install express`, developers can create a basic server that listens on a specific port (commonly port 3000) and responds to client requests.

For the Game Store project, three primary routes can be defined:

- **Home Route (/):** Displays the homepage with featured games, latest releases, or promotions.
- **About Route (/about):** Provides details about the Game Store, its purpose, and its development background.
- **Contact Route (/contact):** Contains information or forms for users to connect with the store for feedback, game inquiries, or customer support.

These routes can be easily implemented using Express methods like `app.get()`, which maps a specific URL path to a callback function that sends a response to the user. Developers can also serve HTML files or templates from a **public** directory to enhance the presentation layer.

This foundational setup ensures that the web application is modular, maintainable, and ready for expansion — such as adding a **database for game data**, **user authentication**, or **payment integration** in later stages.

## Code-

### [Server.js](#)

```
const express = require('express');
const path = require('path');
const bodyParser = require('body-parser');
const connection = require('./db'); // Ensure db.js exports a MySQL/MariaDB connection
const app = express();

// =====
// ⚙️ VIEW & STATIC SETUP
// =====

app.set('views', path.join(__dirname, 'public'));
app.set('view engine', 'ejs');
app.use(express.static(path.join(__dirname, 'public')));
app.use(bodyParser.urlencoded({ extended: true }));
app.use(bodyParser.json());

// =====
// 🌐 MAIN ROUTES
// =====

app.get('/', (req, res) => res.render('index', { pageTitle: 'Home | GameZone' }));
app.get('/about', (req, res) => res.render('about', { pageTitle: 'About | GameZone' }));
app.get('/contact', (req, res) => res.render('contact', { pageTitle: 'Contact | GameZone' }));
app.get('/cart', (req, res) => res.render('cart', { pageTitle: 'Cart | GameZone' }));
app.get('/shop', (req, res) => res.render('shop', { pageTitle: 'Shop | GameZone' }));
app.get('/login', (req, res) => res.render('login', { pageTitle: 'Login | GameZone' }));
app.get('/register', (req, res) => res.render('register', { pageTitle: 'Register | GameZone' }));

// =====
// 📝 REGISTER ROUTE
// =====

app.post('/register', (req, res) => {
  const { username, email, phone, dob, genre, password } = req.body;

  if (!username || !email || !phone || !dob || !genre || !password) {
    return res.status(400).json({ message: '⚠️ All fields are required.' });
  }

  const sql = 'INSERT INTO users (username, email, phone, dob, genre, password) VALUES (?, ?, ?, ?, ?, ?)';
  connection.query(sql, [username, email, phone, dob, genre, password], (err, result) => {
    if (err) {
```

```

    console.error('✗ Error inserting data:', err);
    return res.status(500).json({ message: 'Database error. Please try again.' });
  }
  console.log('☑ User registered successfully:', result.insertId);
  res.json({ message: 'User registered successfully!' });
});
// =====
// 🔗 LOGIN ROUTE
// =====
app.post('/login', (req, res) => {
  const { email, password } = req.body;

  if (!email || !password) {
    return res.status(400).json({ message: '⚠ Email and password are required.' });
  }
  const sql = 'SELECT * FROM users WHERE email = ? AND password = ?';
  connection.query(sql, [email, password], (err, results) => {
    if (err) {
      console.error('✗ Error checking login:', err);
      return res.status(500).json({ message: 'Server error. Please try again.' });
    }
    if (results.length > 0) {
      console.log('☑ ${email} logged in successfully!');
      res.json({ message: 'Login successful!' });
    } else {
      res.json({ message: '✗ Invalid email or password!' });
    }
  });
});
// =====
// 🚀 START SERVER
// =====
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log('☑ GameZone server running at http://localhost:${PORT}'));

```

## Npm Installation-

```
PS C:\Users\sujit\Demo> npm install express

added 68 packages, and audited 69 packages in 2s

16 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\sujit\Demo> S
```

```
> node_modules
{} package-lock.json
{} package.json
```

```
PS C:\Users\sujit\Demo> npm init
This utility will walk you through creating a package.json file.
It only covers the most common items, and tries to guess sensible defaults.

See `npm help init` for definitive documentation on these fields
and exactly what they do.

Use `npm install <pkg>` afterwards to install a package and
save it as a dependency in the package.json file.

Press ^C at any time to quit.
package name: (demo)
version: (1.0.0)
description:
entry point: (index.js)
test command:
git repository:
keywords:
keywords:
keywords:
keywords:
author:
license: (ISC)
type: (commonjs)
About to write to C:\Users\sujit\Demo\package.json:

keywords:
author:
license: (ISC)
type: (commonjs)
About to write to C:\Users\sujit\Demo\package.json:

{
  "name": "demo",
  keywords:
  author:
  license: (ISC)
  type: (commonjs)
  About to write to C:\Users\sujit\Demo\package.json:

  license: (ISC)
  type: (commonjs)
  About to write to C:\Users\sujit\Demo\package.json:

  type: (commonjs)
  About to write to C:\Users\sujit\Demo\package.json:
```

## **Conclusion-**

Setting up a Node.js project with an Express server for a Game store establishes the backbone for a scalable and interactive web application. By defining routes like Home, About, and Contact, the project achieves clear navigation and structure, ensuring a smooth user experience. The combination of Node.js's asynchronous efficiency and Express.js's simplicity makes this setup ideal for developing high-performance, modern web platforms.

## Experiment No-04

### Problem Statement-

The objective of this project is to develop a Game Store web application that demonstrates server-side scripting and database integration for dynamic content management. The system aims to handle core functionalities such as displaying game listings, managing user data, and processing user interactions efficiently through Node.js and Express.js on the backend. By connecting the application to a database (such as MongoDB or MySQL), the Game Store will be able to store, retrieve, update, and delete game details, user information, and purchase history dynamically. The project highlights how server-side scripting can enhance performance, maintain security, and enable smooth data communication between the client and the server.

### Theory-

In traditional web applications, pages were static and required manual updates to reflect any change. However, modern applications rely on **server-side scripting** to deliver **dynamic content**, handle **user inputs**, and manage **real-time interactions**. Server-side scripting refers to code execution on the web server rather than on the client's browser. It plays a vital role in making applications interactive, secure, and capable of handling complex operations such as authentication, database connectivity, and data manipulation.

### 1. Server-Side Scripting Overview

Server-side scripting is used to create web applications that respond dynamically to user actions. When a user sends a request (like viewing a game or logging in), the server processes that request using a scripting language, interacts with a database if needed, and sends back the appropriate response (HTML, JSON, or rendered page). This ensures that the content shown to the user is always up-to-date and customized based on real-time data.

Languages such as **JavaScript (Node.js)**, **PHP**, **Python (Django, Flask)**, and **Ruby (on Rails)** are commonly used for server-side scripting. In this project, **Node.js** is chosen due to its performance, scalability, and compatibility with JavaScript across both frontend and backend, ensuring a seamless development process.

### 2. Node.js and Express.js

Node.js is a runtime environment that allows JavaScript to be executed outside the browser. It uses an asynchronous, event-driven architecture, which means it can handle multiple client requests simultaneously without blocking the execution thread. This makes Node.js perfect for applications that require high performance, such as online game stores where multiple users might be browsing and purchasing games at the same time.

To simplify the creation of server routes and APIs, Express.js is used as the web framework. Express provides a minimal and flexible structure for handling HTTP requests (GET, POST,

PUT, DELETE), routing, middleware integration, and response management. It allows developers to build scalable APIs and serve static files like images, CSS, or game cover art efficiently.

For example, in the Game Store application:

- The Home route (/) displays the list of available games fetched from the database.
- The About route (/about) explains the vision and purpose of the store.
- The Contact route (/contact) allows users to reach out for inquiries or support.

### 3. Database Integration

The core of dynamic web applications lies in their ability to store and retrieve data. Databases act as the backbone of such systems, maintaining structured and persistent data. In this Game Store application, the database can store:

- Game details (name, category, price, platform, release date, rating, and description)
- User details (name, email, password, and purchase history)
- Orders and transaction records

Depending on the project requirements, developers can use:

- MongoDB, a NoSQL database that stores data in a flexible JSON-like format.
- MySQL, a relational database that organizes data in tables using structured queries (SQL).

Using Mongoose (for MongoDB) or Sequelize (for MySQL), developers can define models to interact with the database efficiently. These Object Data Modeling (ODM/ORM) tools help manage data relationships and validations seamlessly.

For instance:

- When a new game is added, a POST request inserts it into the database.
- When a user browses the store, a GET request retrieves the list of available games.
- When a user edits their profile, a PUT request updates the data.



- When an item is deleted from inventory, a DELETE request removes it.

#### 4. Security and Middleware

Security is a fundamental concern in any web application, especially one involving users and purchases. Express middleware allows developers to handle authentication, authorization, and input validation. Common middleware includes:

- body-parser for handling form data,
- helmet for securing HTTP headers,
- express-session or JWT (JSON Web Token) for user authentication.

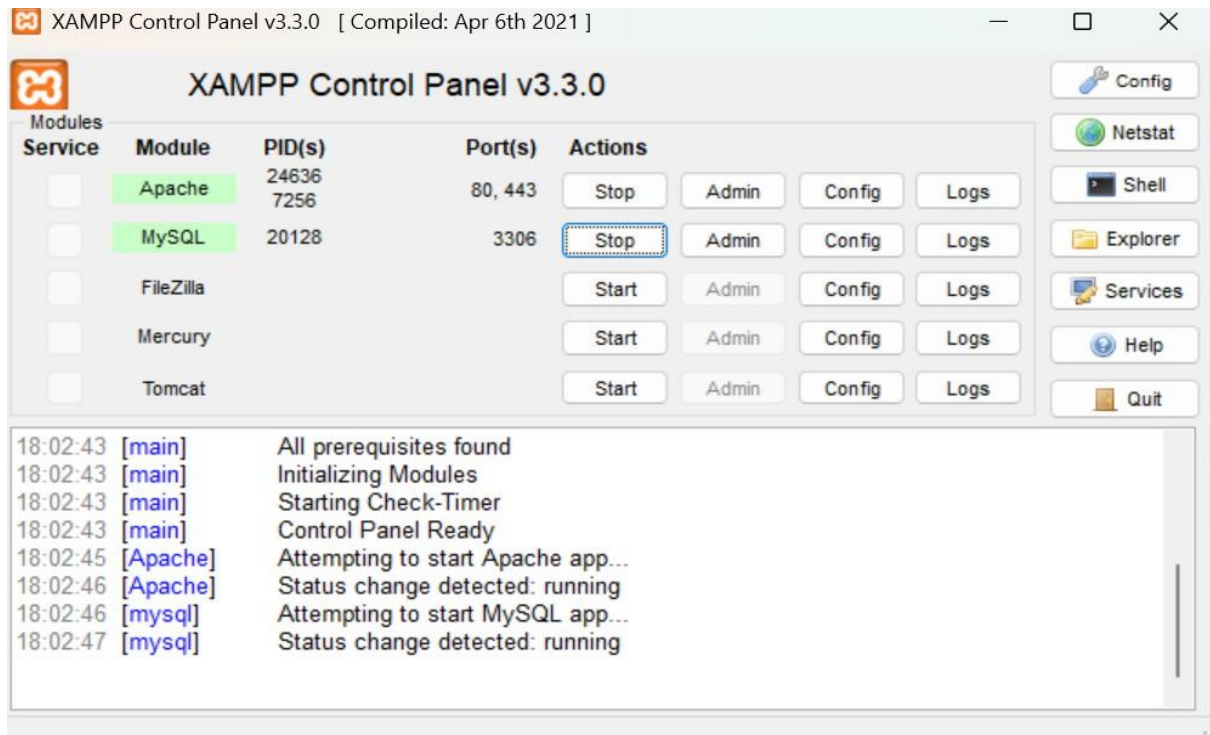
By implementing encryption (bcrypt.js) for storing passwords and input validation to prevent SQL/NoSQL injections, the Game Store ensures data integrity and security.

#### 5. Web Application Workflow

When a user accesses the Game Store:

1. The request is sent to the Express server.
2. The server routes the request to the appropriate handler (e.g., /, /about, /contact, /games, /login).
3. The server may fetch data from the database if required.
4. The server processes the response and sends back a rendered HTML page or a JSON response to the browser.

**Code-**



## Server.js

```
const express = require('express');
const path = require('path');
const bodyParser = require('body-parser');
const connection = require('./db'); // Ensure db.js exports a MySQL/MariaDB connection
const app = express();

// =====
// ⚙️ VIEW & STATIC SETUP
// =====

app.set('views', path.join(__dirname, 'public'));
app.set('view engine', 'ejs');
app.use(express.static(path.join(__dirname, 'public')));
app.use(bodyParser.urlencoded({ extended: true }));
app.use(bodyParser.json());

// =====
// 🌐 MAIN ROUTES
// =====

app.get('/', (req, res) => res.render('index', { pageTitle: 'Home | GameZone' }));
app.get('/about', (req, res) => res.render('about', { pageTitle: 'About | GameZone' }));
app.get('/contact', (req, res) => res.render('contact', { pageTitle: 'Contact | GameZone' }));
app.get('/cart', (req, res) => res.render('cart', { pageTitle: 'Cart | GameZone' }));
app.get('/shop', (req, res) => res.render('shop', { pageTitle: 'Shop | GameZone' }));
app.get('/login', (req, res) => res.render('login', { pageTitle: 'Login | GameZone' }));
```

```

app.get('/register', (req, res) => res.render('register', { pageTitle: 'Register | GameZone' }));

// =====
// 🌀 REGISTER ROUTE
// =====

app.post('/register', (req, res) => {
  const { username, email, phone, dob, genre, password } = req.body;

  if (!username || !email || !phone || !dob || !genre || !password) {
    return res.status(400).json({ message: '⚠️ All fields are required.' });
  }

  const sql = 'INSERT INTO users (username, email, phone, dob, genre, password) VALUES (?, ?, ?, ?, ?, ?)';
  connection.query(sql, [username, email, phone, dob, genre, password], (err, result) => {
    if (err) {
      console.error('❌ Error inserting data:', err);
      return res.status(500).json({ message: 'Database error. Please try again.' });
    }

    console.log('✅ User registered successfully:', result.insertId);
    res.json({ message: 'User registered successfully!' });
  });
});

// =====
// 🔑 LOGIN ROUTE
// =====

app.post('/login', (req, res) => {
  const { email, password } = req.body;

  if (!email || !password) {
    return res.status(400).json({ message: '⚠️ Email and password are required.' });
  }

  const sql = 'SELECT * FROM users WHERE email = ? AND password = ?';
  connection.query(sql, [email, password], (err, results) => {
    if (err) {
      console.error('❌ Error checking login:', err);
      return res.status(500).json({ message: 'Server error. Please try again.' });
    }
  });
});

```

```

if (results.length > 0) {
  console.log('✅ ${email} logged in successfully!');
  res.json({ message: 'Login successful!' });
} else {
  res.json({ message: '❌ Invalid email or password!' });
}
});
});

// =====
// 🚀 START SERVER
// =====

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log('✅ GameZone server running at http://localhost:${PORT}'));

```

## Output-

```

...
❖ PS C:\Users\sujit\game-store> npm start

> game-store@1.0.0 start
> node server.js

✅ GameZone server running at http://localhost:3000
✅ Connected to MySQL database

```

SELECT \* FROM `users`

☐ Profiling [\[ Edit inline \]](#) [\[ Edit \]](#) [\[ Explain SQL \]](#) [\[ Create PHP code \]](#) [\[ Refresh \]](#)

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

|                          |                  | id | username | email                     | phone      | dob        | genre     | password   |
|--------------------------|------------------|----|----------|---------------------------|------------|------------|-----------|------------|
| <input type="checkbox"/> | Edit Copy Delete | 1  | sujit    | sumigood@gmail.com        | 9309115544 | 2006-09-19 | Adventure | Sujit8090  |
| <input type="checkbox"/> | Edit Copy Delete | 2  | sujit    | sumigood@gmail.com        | 9309115544 | 2006-09-19 | Adventure | Sujit8090  |
| <input type="checkbox"/> | Edit Copy Delete | 3  | sujit2   | sumigood@gmail.com        | 9309115544 | 2006-09-19 | Sports    | Sujit80901 |
| <input type="checkbox"/> | Edit Copy Delete | 4  | sujit    | sumigood@gmail.com        | 9309113344 | 2025-11-28 | Strategy  | Sujit80901 |
| <input type="checkbox"/> | Edit Copy Delete | 5  | sujit    | sujitbhagwat411@gmail.com | 9309115544 | 2025-11-10 | Adventure | Sujit80901 |

☐ Check all | With selected: Edit Copy Delete Export

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Query results operations

Print Copy to clipboard Export Display chart Create view

Bookmark this SQL query

## Create Your Account

Username

Email

Phone Number

Date of Birth

Favorite Game Genre

Password

Minimum 6 characters

Confirm Password

Register

### Conclusion-

Developing a Game Store web application using server-side scripting and database integration provides a comprehensive understanding of backend web technologies. By leveraging Node.js and Express.js, the system achieves high performance, modularity, and scalability, while database integration ensures persistent data management for games, users, and transactions.

|                                                 |                             |                                  |
|-------------------------------------------------|-----------------------------|----------------------------------|
| <b>Prepared By – TTT Coordination Committee</b> |                             |                                  |
|                                                 |                             |                                  |
|                                                 |                             |                                  |
|                                                 |                             |                                  |
| <b>Prof. Rohini Bhosale</b>                     | <b>Prof. Rohini Bhosale</b> | <b>Prof. Dr. Prashant Dhotre</b> |
| <b>Subject Teacher</b>                          | <b>Course Coordinator</b>   | <b>Head - Department of CSE</b>  |
|                                                 |                             |                                  |
|                                                 |                             |                                  |

## Problem Statement -

1. Create a Static Homepage Layout Using Bootstrap

1.1 Design a responsive homepage using Bootstrap components like container, jumbotron, headers, and buttons.

1.2 Use the grid system to create a structured and adaptive layout suitable for any web app

## Theory-

### Introduction to Bootstrap

Bootstrap is an open-source, front-end framework developed by Twitter that allows developers to design responsive and mobile-first websites quickly. It includes a comprehensive collection of HTML, CSS, and JavaScript components that make it easier to build modern user interfaces.

Bootstrap simplifies layout creation through its **12-column grid system**, responsive utilities, and pre-styled UI elements like **buttons, forms, navigation bars, modals, and cards**. It ensures consistency across browsers and devices, drastically reducing development time.

### 1. Responsive Web Design

Responsive design ensures that a webpage automatically adjusts its layout and elements to fit different screen sizes — desktops, tablets, and mobile devices. Bootstrap achieves responsiveness using fluid grids, flexible images, and media queries.

Bootstrap uses four key breakpoints:

- xs (extra small devices like phones)
- sm (small devices like tablets)
- md (medium devices like laptops)
- lg and xl (large and extra-large devices like desktops)

By combining these breakpoints with grid classes, developers can design adaptive layouts efficiently.

### 2. Bootstrap Container

The **container** is the base element that wraps the entire content and provides proper alignment and padding.

Bootstrap offers:

- `.container` – a fixed-width container that adjusts with breakpoints.
- `.container-fluid` – a full-width container spanning the entire screen width.

Example:

```
<div class="container">
```

```
<h1>Welcome to Our Homepage</h1>
```

```
</div>
```

### 3. Grid System

The **grid system** divides the page into 12 columns. Developers can combine columns using classes like `.col-md-4` or `.col-sm-6` to control how many columns each section takes up on different devices.

Example:

```
<div class="row">

<div class="col-md-4">Section 1</div>

<div class="col-md-4">Section 2</div>

<div class="col-md-4">Section 3</div>

</div>
```

This ensures a flexible and responsive structure

### 4. Jumbotron (Hero Section)

A **jumbotron** (or “hero section” in newer versions) is used to highlight key information like a welcome message or tagline. It draws user attention and often includes a call-to-action button.

Example:

```
<div class="jumbotron text-center bg-light">

<h1>Welcome to Our Web App</h1>

<p>Building responsive designs with ease using Bootstrap</p>

<a href="#" class="btn btn-primary">Get Started</a>

</div>
```

### 5. Headers and Buttons

Headers (`<h1>` to `<h6>`) establish content hierarchy, while **Bootstrap buttons** (`.btn`, `.btn-primary`, `.btn-outline-success`, etc.) enhance interactivity with predefined styles.

These elements combine to create an attractive and user-friendly homepage, maintaining design consistency across all devices.

### Code-



## Index.html-

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8" />

  <meta name="viewport" content="width=device-width, initial-scale=1.0" />

  <title><%= pageTitle %></title>

  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet" />

  <link href="https://unpkg.com/aos@2.3.1/dist/aos.css" rel="stylesheet">

  <link href="https://fonts.googleapis.com/css2?family=Poppins:wght@400;600;700&display=swap"
rel="stylesheet" />

  <style>

    body {

      font-family: 'Poppins', sans-serif;

      background-color: #0e0e0e;

      color: #f8f9fa;

      overflow-x: hidden;

      scroll-behavior: smooth;

    }

    .navbar {

      background: rgba(20, 20, 20, 0.95);

      backdrop-filter: blur(10px);

      box-shadow: 0 4px 25px rgba(0, 0, 0, 0.5);

      transition: background 0.4s ease;
```

```
}

.navbar-brand {

  font-size: 1.6rem;

  color: #00aaff !important;

  font-weight: 700;

}

.nav-link {

  color: #f8f9fa !important;

  margin: 0 8px;

  font-weight: 500;

  transition: color 0.3s ease;

}

.nav-link:hover {

  color: #00aaff !important;

}

header {

  background: linear-gradient(180deg, #111, #222);

  padding: 6rem 1rem;

  text-align: center;

  color: #fff;

}

header h1 {

  font-weight: 700;

  font-size: 3rem;

  text-shadow: 0 3px 10px rgba(0, 170, 255, 0.4);
```

```
}

header p {

  font-size: 1.2rem;

  margin-bottom: 1.5rem;

  opacity: 0.9;

}

header .btn {

  background-color: #00aaff;

  border: none;

  font-weight: 600;

  transition: transform 0.3s ease, background 0.3s ease;

}

header .btn:hover {

  transform: scale(1.1);

  background-color: #007acc;

}

section h2 {

  color: #00aaff;

  font-weight: 700;

  margin-bottom: 2rem;

  text-align: center;

}

.game-card {

  background: #1b1b1b;

  border: none;
```

```
border-radius: 14px;

transition: transform 0.4s ease, box-shadow 0.4s ease;

overflow: hidden;
}

.game-card:hover {

transform: translateY(-10px);

box-shadow: 0 10px 35px rgba(0, 170, 255, 0.4);

}

.game-card img {

border-radius: 14px 14px 0 0;

}

.news-card {

background: #1a1a1a;

border-radius: 12px;

padding: 1.5rem;

height: 100%;

transition: transform 0.4s ease, background 0.3s ease;

}

.news-card:hover {

transform: translateY(-8px);

background: #232323;

}

.news-card h4 {

color: #00aaff;

font-weight: 600;
```

```
margin-bottom: 0.6rem;

}

.stats {

background: linear-gradient(90deg, #0e0e0e, #1c1c1c);

padding: 4rem 1rem;

border-top: 1px solid #222;

border-bottom: 1px solid #222;

}

.stats h3 {

font-size: 2.4rem;

color: #00aaff;

}

.stats p {

color: #bbb;

font-size: 1rem;

}

footer {

background: #1b1b1b;

color: #bbb;

padding: 1.5rem 0;

text-align: center;

font-size: 0.9rem;

}

[data-aos] {

transition-property: transform, opacity;
```

```

    transition-duration: 1s;

}

@media (max-width: 992px) {

    header h1 { font-size: 2.2rem; }

    header p { font-size: 1rem; }

}

@media (max-width: 576px) {

    header { padding: 3rem 1rem; }

    header h1 { font-size: 1.8rem; }

}

</style>

</head>

<body>

<nav class="navbar navbar-expand-lg navbar-dark fixed-top">

    <div class="container">

        <a class="navbar-brand" href="/"><img alt="GameZone logo" data-bbox="422 594 442 608"/> GameZone</a>

        <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navbarNav">

            <span class="navbar-toggler-icon"></span>

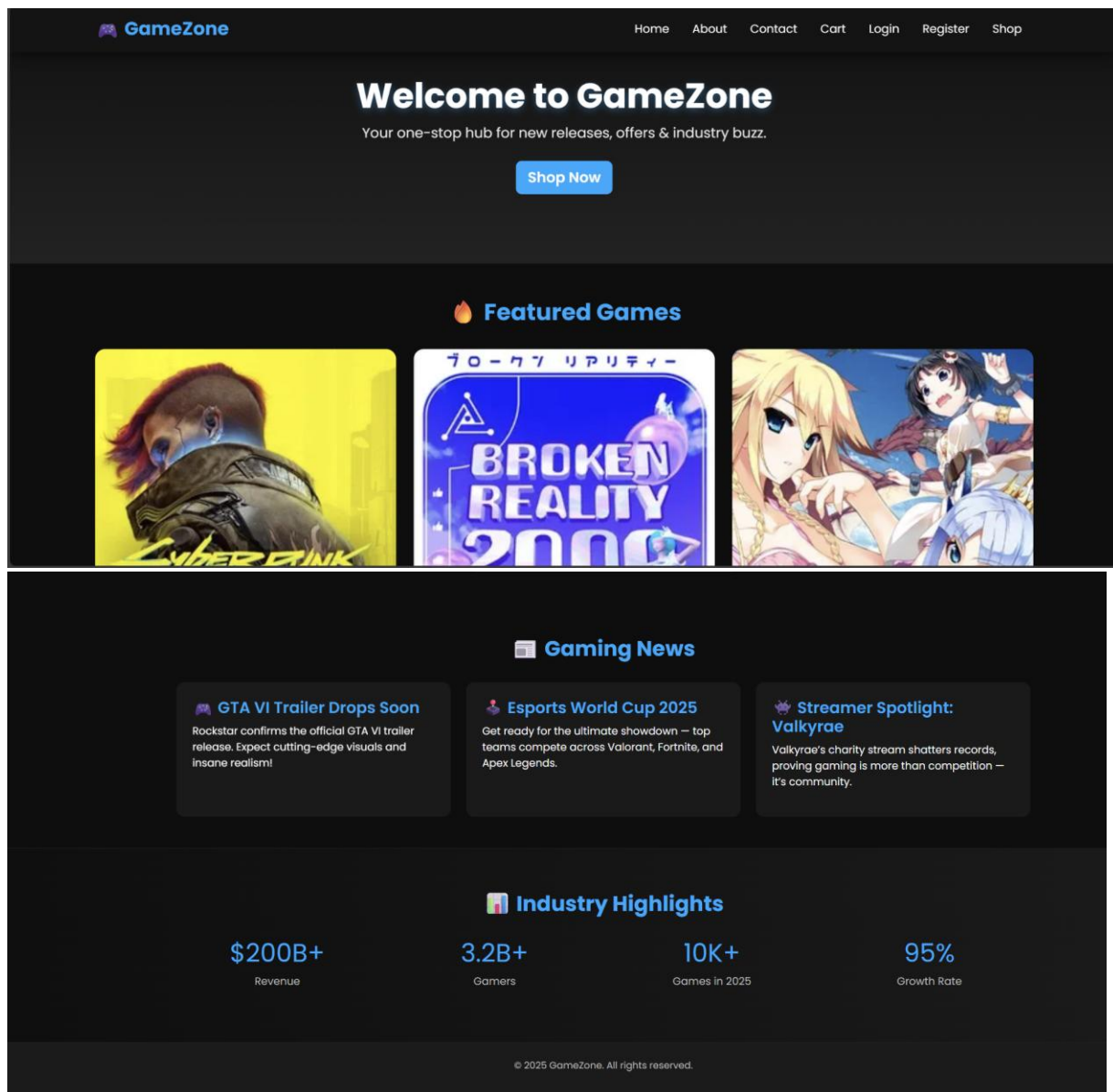
        </button>

        <div class="collapse navbar-collapse justify-content-end" id="navbarNav">

            <ul class="navbar-na

```

Output-



## Conclusion-

By designing a static homepage layout using Bootstrap's responsive features and grid system, developers can achieve a visually consistent and device-friendly interface. The implementation reduces design complexity, enhances user experience, and maintains a professional and modern look across all screen resolutions—making Bootstrap an ideal choice for building adaptive web application homepages.

## Experiment No.2

### Problem Statement-

To create a grid-based information page using Bootstrap's Grid system to display multiple content blocks like products or blog entries with proper alignment and spacing.

### Theory-

Bootstrap, developed by Twitter, is one of the most popular **front-end frameworks** for building responsive, mobile-first web applications. At its core lies the **Grid System**, a flexible layout model based on a **12-column structure**. This system allows developers to divide the webpage into rows and columns, enabling precise control over content alignment and distribution. Each column is represented using classes like `.col-sm-6`, `.col-md-4`, `.col-lg-3`, etc., which automatically adjust the number of visible columns depending on the screen size.

For example, using `.col-md-4` divides a row into three equal columns on medium-sized screens, while `.col-sm-6` creates two columns on smaller screens. This adaptability ensures that the webpage remains **user-friendly** and visually consistent across all devices.

To enhance the layout, Bootstrap offers a range of **predefined components**—most notably **cards**—that encapsulate information in small, neatly designed containers. A card typically includes an image, title, description, and a call-to-action button. Cards are ideal for representing products, blog posts, user profiles, or feature highlights in a structured grid layout. The use of Bootstrap's **utility classes** like `p-3` (padding), `m-2` (margin), and `text-center` (alignment) further enhances the design by ensuring proper spacing, readability, and alignment.

In addition, the grid system allows for **nesting**, where columns can contain rows within them to create more complex layouts. This makes it easier to organize diverse types of content without breaking the design's flow. By combining the grid with components like **containers**, **rows**, and **columns**, developers can build layouts that are not only visually appealing but also semantically structured and accessible.

Moreover, the use of **responsive design principles** ensures that the content automatically adjusts to any device's screen width, eliminating the need for separate designs for different platforms. This approach improves performance, accessibility, and user satisfaction.

### Code-



## About.html-

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>About Us - GameZone</title>
  <link
    href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
    rel="stylesheet"
  />
  <style>
    body {
      font-family: 'Poppins', sans-serif;
      background: linear-gradient(135deg, #1b1b1b, #2d2d2d);
      color: #fff;
      overflow-x: hidden;
    }
    .navbar {
      box-shadow: 0 4px 10px rgba(0,0,0,0.5);
      transition: background 0.4s ease;
    }
    .navbar:hover {
      background: rgba(0,0,0,0.95);
    }

    .nav-link {
      position: relative;
      font-weight: 500;
      transition: all 0.3s ease;
    }
    .nav-link:hover {
      color: #0d6efd !important;
    }
    .nav-link::after {
      content: "";
      position: absolute;
      width: 0%;
      height: 2px;
      background: #0d6efd;
      left: 0;
```

```
    bottom: 0;
    transition: width 0.3s ease;
  }
.nav-link:hover::after {
  width: 100%;
}
header {
  background: linear-gradient(135deg, #0d6efd, #6610f2);
  color: #fff;
  text-align: center;
  padding: 6rem 1rem;
  animation: fadeInDown 1s ease-in-out;
}
header h1 {
  font-size: 3rem;
  font-weight: 700;
  animation: slideIn 1s ease-in-out;
}
header p {
  font-size: 1.2rem;
  opacity: 0.9;
  animation: fadeIn 1.5s ease-in-out;
}
section.container {
  animation: fadeUp 1.5s ease-in-out;
}

.img-fluid {
  border-radius: 15px;
  box-shadow: 0 0 25px rgba(0,0,0,0.4);
  transition: transform 0.3s ease, box-shadow 0.3s ease;
}
.img-fluid:hover {
  transform: scale(1.03);
  box-shadow: 0 0 35px rgba(13,110,253,0.4);
}
.btn-primary {
  background: linear-gradient(135deg, #0d6efd, #6610f2);
  border: none;
  transition: transform 0.3s ease, background 0.3s ease;
  font-weight: 600;
}
```

```

.btn-primary:hover {
  background: linear-gradient(135deg, #6610f2, #0d6efd);
  transform: scale(1.05);
}

.bg-light {
  background: rgba(255, 255, 255, 0.05) !important;
  backdrop-filter: blur(10px);
  color: #fff;
}

.border {
  border: 1px solid rgba(255, 255, 255, 0.2) !important;
}

.shadow-sm {
  box-shadow: 0 4px 20px rgba(0,0,0,0.3) !important;
  transition: transform 0.3s ease, box-shadow 0.3s ease;
}

.shadow-sm:hover {
  transform: translateY(-5px);
  box-shadow: 0 6px 25px rgba(13,110,253,0.4) !important;
}

footer {
  background: #000;
  color: #ccc;
  font-size: 0.9rem;
  letter-spacing: 0.5px;
  animation: fadeIn 2s ease-in-out;
}

@keyframes fadeIn {
  from { opacity: 0; }
  to { opacity: 1; }
}

@keyframes fadeInDown {
  from {
    opacity: 0;
    transform: translateY(-40px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}

```

```

@keyframes fadeUp {
  from {
    opacity: 0;
    transform: translateY(50px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}

@keyframes slideIn {
  from {
    opacity: 0;
    transform: translateX(-50px);
  }
  to {
    opacity: 1;
    transform: translateX(0);
  }
}

@media (max-width: 768px) {
  header h1 {
    font-size: 2rem;
  }
  .img-fluid {
    margin-bottom: 1.5rem;
  }
}
</style>
</head>
<body>
<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
  <div class="container">
    <a class="navbar-brand fw-bold" href="/"> 🎮 GameZone </a>
    <button
      class="navbar-toggler"
      type="button"
      data-bs-toggle="collapse"
      data-bs-target="#navbarNav"
      title="Toggle navigation"
      aria-label="Toggle navigation"
    >

```

```

    <span class="navbar-toggler-icon"></span>
  </button>
  <div class="collapse navbar-collapse" id="navbarNav">
    <ul class="navbar-nav ms-auto">
      <li class="nav-item"><a class="nav-link" href="/">Home</a></li>
      <li class="nav-item"><a class="nav-link active" href="/about">About</a></li>
      <li class="nav-item"><a class="nav-link" href="/contact">Contact</a></li>
      <li class="nav-item"><a class="nav-link" href="/cart">Cart</a></li>
      <li class="nav-item"><a class="nav-link" href="/login">Login</a></li>
      <li class="nav-item"><a class="nav-link" href="/register">Register</a></li>
    </ul>
  </div>
</div>
</nav>

<header>
  <div class="container">
    <h1 class="fw-bold">About GameZone</h1>
    <p class="lead">Your Ultimate Destination for Gaming Adventures</p>
  </div>
</header>

<section class="container py-5">
  <div class="row align-items-center g-4">
    <div class="col-md-6 text-center">
      
    </div>
    <div class="col-md-6">
      <h2 class="fw-bold mb-3">Who We Are</h2>
      <p>
        At <strong>GameZone</strong>, we are passionate gamers who believe in
        bringing the best gaming experiences to everyone. Our store features a
        wide range of the latest releases, timeless classics, and exclusive
        titles — all in one place.
      </p>
    </div>
  </div>

```

```

<p>
  Whether you are into action, adventure, sports, or strategy games, we
  have something for everyone. We're committed to offering affordable
  prices, secure shopping, and the best customer support.
</p>
<a href="/contact" class="btn btn-primary">Get in Touch</a>
</div>
</div>
</section>

<section class="bg-light py-5">
  <div class="container text-center">
    <h2 class="fw-bold mb-4">Our Mission & Vision</h2>
    <div class="row g-4">
      <div class="col-md-6">
        <div class="p-4 border rounded shadow-sm h-100">
          <h4 class="fw-bold">🎮 Mission</h4>
          <p>
            To provide gamers worldwide with easy access to the best games,
            seamless shopping experience, and trustworthy service.
          </p>
        </div>
      </div>
      <div class="col-md-6">
        <div class="p-4 border rounded shadow-sm h-100">
          <h4 class="fw-bold">🌐 Vision</h4>
          <p>
            To become the leading global online game store, where every gamer
            finds their next adventure.
          </p>
        </div>
      </div>
    </div>
  </div>
</section>

<footer class="py-3 text-center">
  <p class="mb-0">&copy; 2025 GameZone. All rights reserved 🎮</p>
</footer>
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
<script>
  document.addEventListener("DOMContentLoaded", () => {

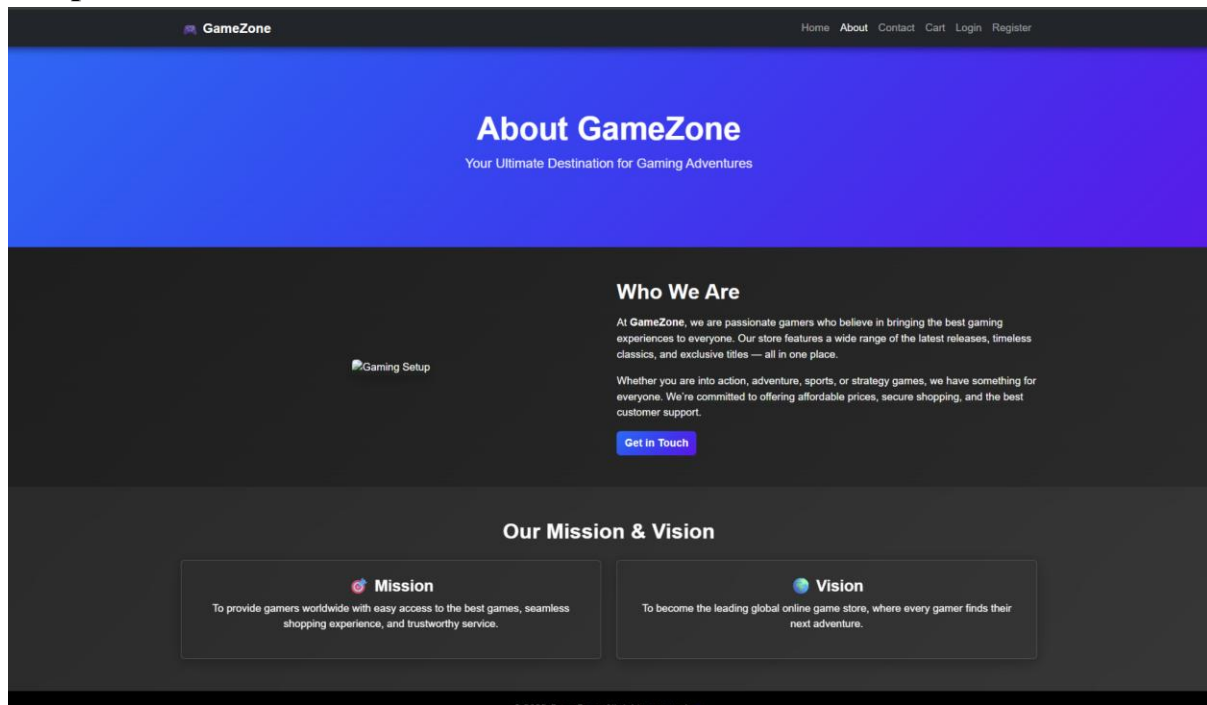
```

```

    console.log("Welcome to GameZone — where every click starts a new adventure!");
  });
</script>
</body>
</html>

```

## Output-



## Cart.html-

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Cart - GameZone</title>

  <!-- Bootstrap CSS -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet" />
  <link href="https://cdnjs.cloudflare.com/ajax/libs/animate.css/4.1.1/animate.min.css" rel="stylesheet" />

  <style>
    body {
      background-color: #121212;
      color: #fff;
      font-family: "Poppins", sans-serif;
    }
  </style>

```

```
.navbar {  
  box-shadow: 0 4px 8px rgba(0,0,0,0.4);  
}
```

```
.navbar-brand {  
  font-size: 1.5rem;  
  font-weight: bold;  
  letter-spacing: 1px;  
}
```

```
.nav-link {  
  color: #ccc !important;  
  margin: 0 10px;  
  transition: 0.3s;  
}
```

```
.nav-link:hover {  
  color: #0dcaf0 !important;  
  transform: scale(1.05);  
}
```

```
h1 {  
  text-align: center;  
  margin-bottom: 2rem;  
  font-weight: 700;  
  letter-spacing: 1px;  
  animation: fadeInDown 1s;  
}
```

```
#cart-container {  
  background: #f1f1f1;  
  border-radius: 15px;  
  padding: 2rem;  
  box-shadow: 0 0 15px rgba(0,0,0,0.6);  
  animation: fadeInUp 1.2s;  
}
```

```
.cart-item {  
  display: flex;  
  align-items: center;  
  justify-content: space-between;
```



```
background: #2c2c2c;
padding: 15px;
margin-bottom: 10px;
border-radius: 10px;
transition: all 0.3s ease;
}

.cart-item:hover {
background: #343434;
transform: translateY(-3px);
}

.cart-item img {
width: 80px;
height: 80px;
border-radius: 10px;
margin-right: 15px;
}

.item-info {
flex: 1;
}

.total-section {
text-align: right;
margin-top: 1.5rem;
}

.btn-checkout {
background: linear-gradient(90deg, #0dcaf0, #6610f2);
border: none;
padding: 10px 25px;
border-radius: 30px;
color: #fff;
font-weight: bold;
transition: 0.3s;
}

.btn-checkout:hover {
transform: scale(1.05);
opacity: 0.9;
}
```

```

    footer {
      background: #0d0d0d;
      margin-top: 3rem;
      padding: 1rem;
      color: #bbb;
    }
  </style>
</head>
<body>
  <nav class="navbar navbar-expand-lg navbar-dark bg-dark py-3">
    <div class="container">
      <a class="navbar-brand" href="/">🎮 GameZone</a>
      <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target="#navMenu">
        <span class="navbar-toggler-icon"></span>
      </button>
      <div class="collapse navbar-collapse justify-content-end" id="navMenu">
        <ul class="navbar-nav">
          <li class="nav-item"><a class="nav-link" href="/">Home</a></li>
          <li class="nav-item"><a class="nav-link" href="/about">About</a></li>
          <li class="nav-item"><a class="nav-link" href="/contact">Contact</a></li>
          <li class="nav-item"><a class="nav-link active text-info" href="/cart">Cart</a></li>
          <li class="nav-item"><a class="nav-link" href="/login">Login</a></li>
          <li class="nav-item"><a class="nav-link" href="/register">Register</a></li>
        </ul>
      </div>
    </div>
  </nav>

  <!-- Cart Section -->
  <main class="container my-5">
    <div id="cart-container">
      <h1>Your Cart 🛒</h1>
      <div id="cart-items">
        <div class="cart-item">
          
          <div class="item-info">
            <h5>Counter-Strike 2</h5>
            <p class="mb-0 text-info">$19.99</p>
          </div>
          <button class="btn btn-sm btn-danger">Remove</button>
        </div>
      </div>
    </div>
  </main>

```

```

<div class="cart-item">
  
  <div class="item-info">
    <h5>GTA V</h5>
    <p class="mb-0 text-info">$29.99</p>
  </div>
  <button class="btn btn-sm btn-danger">Remove</button>
</div>
</div>

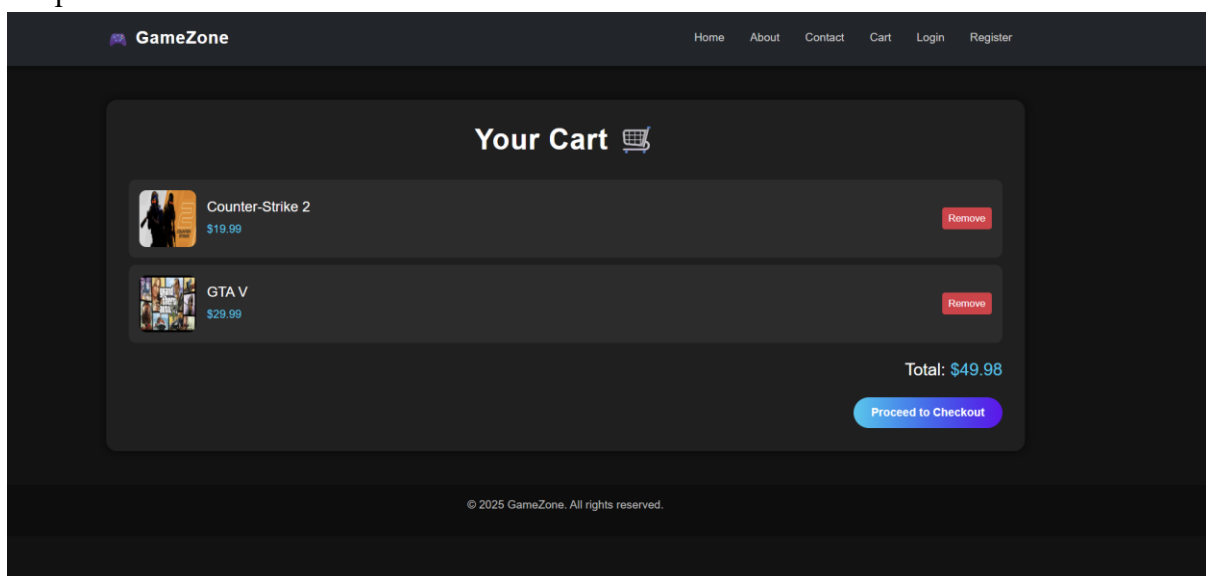
<div class="total-section">
  <h4>Total: <span class="text-info">$49.98</span></h4>
  <button class="btn-checkout mt-3">Proceed to Checkout</button>
</div>
</div>
</main>

<footer class="text-center">
  <p>&copy; 2025 GameZone. All rights reserved.</p>
</footer>

<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>
</body>
</html>

```

## Output-



## Conclusion -

A grid-based information page built using Bootstrap ensures a well-structured, responsive, and visually uniform layout that enhances content presentation. By combining the grid system with components like cards and spacing utilities, developers can deliver a professional and scalable design that maintains alignment, readability, and adaptability across various devices, ensuring an optimal user experience.

## Experiment No-03

### Problem Statement-

To initialize and configure a Node.js project using Express.js to create basic routes like Home, About, and Contact.

### Theory-

Node.js is a **runtime environment** built on Chrome's V8 JavaScript engine that allows developers to run JavaScript on the server side. It enables the development of fast, scalable, and efficient web applications. The core advantage of Node.js lies in its **non-blocking, event-driven architecture**, which allows multiple client requests to be handled concurrently without overloading the system.

To start a Node.js project, developers use the command `npm init`, which initializes the project and creates a `package.json` file containing metadata such as project name, version, scripts, and dependencies. This file serves as the blueprint of the project, managing installed libraries and ensuring version control.

**Express.js**, a minimal and flexible Node.js web framework, is widely used to build web servers and APIs efficiently. It simplifies the process of defining routes, handling HTTP requests, and serving static files. Once Express is installed using `npm install express`, developers can create a basic server that listens on a specific port (commonly port 3000) and responds to client requests.

For the Game Store project, three primary routes can be defined:

- **Home Route (/):** Displays the homepage with featured games, latest releases, or promotions.
- **About Route (/about):** Provides details about the Game Store, its purpose, and its development background.
- **Contact Route (/contact):** Contains information or forms for users to connect with the store for feedback, game inquiries, or customer support.

These routes can be easily implemented using Express methods like `app.get()`, which maps a specific URL path to a callback function that sends a response to the user. Developers can also serve HTML files or templates from a **public** directory to enhance the presentation layer.

This foundational setup ensures that the web application is modular, maintainable, and ready for expansion — such as adding a **database for game data**, **user authentication**, or **payment integration** in later stages.

### Code-

Server.js

```

const express = require('express');
const path = require('path');
const bodyParser = require('body-parser');
const connection = require('./db'); // Ensure db.js exports a MySQL/MariaDB connection
const app = express();

// =====
// ⚙️ VIEW & STATIC SETUP
// =====

app.set('views', path.join(__dirname, 'public'));
app.set('view engine', 'ejs');
app.use(express.static(path.join(__dirname, 'public')));
app.use(bodyParser.urlencoded({ extended: true }));
app.use(bodyParser.json());

// =====
// 🌐 MAIN ROUTES
// =====

app.get('/', (req, res) => res.render('index', { pageTitle: 'Home | GameZone' }));
app.get('/about', (req, res) => res.render('about', { pageTitle: 'About | GameZone' }));
app.get('/contact', (req, res) => res.render('contact', { pageTitle: 'Contact | GameZone' }));
app.get('/cart', (req, res) => res.render('cart', { pageTitle: 'Cart | GameZone' }));
app.get('/shop', (req, res) => res.render('shop', { pageTitle: 'Shop | GameZone' }));
app.get('/login', (req, res) => res.render('login', { pageTitle: 'Login | GameZone' }));
app.get('/register', (req, res) => res.render('register', { pageTitle: 'Register | GameZone' }));

// =====
// 🔑 REGISTER ROUTE
// =====

app.post('/register', (req, res) => {
  const { username, email, phone, dob, genre, password } = req.body;

  if (!username || !email || !phone || !dob || !genre || !password) {
    return res.status(400).json({ message: '⚠️ All fields are required.' });
  }

  const sql = 'INSERT INTO users (username, email, phone, dob, genre, password) VALUES (?, ?, ?, ?, ?, ?)';
  connection.query(sql, [username, email, phone, dob, genre, password], (err, result) => {
    if (err) {
      console.error('❌ Error inserting data:', err);
      return res.status(500).json({ message: 'Database error. Please try again.' });
    }
    console.log('✅ User registered successfully:', result.insertId);
    res.json({ message: 'User registered successfully!' });
  });
});

```

```

});
});
// =====
// 🔑 LOGIN ROUTE
// =====
app.post('/login', (req, res) => {
  const { email, password } = req.body;

  if (!email || !password) {
    return res.status(400).json({ message: '⚠️ Email and password are required.' });
  }
  const sql = 'SELECT * FROM users WHERE email = ? AND password = ?';
  connection.query(sql, [email, password], (err, results) => {
    if (err) {
      console.error('❌ Error checking login:', err);
      return res.status(500).json({ message: 'Server error. Please try again.' });
    }
    if (results.length > 0) {
      console.log('✅ ${email} logged in successfully!');
      res.json({ message: 'Login successful!' });
    } else {
      res.json({ message: '❌ Invalid email or password!' });
    }
  });
});
// =====
// 🚀 START SERVER
// =====
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log('✅ GameZone server running at http://localhost:${PORT}'));

```

## Npm Installation-

```
PS C:\Users\sujit\Demo> npm install express

added 68 packages, and audited 69 packages in 2s

16 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\sujit\Demo> S
```

```
> node_modules
{} package-lock.json
{} package.json
```



```

PS C:\Users\sujit\Demo> npm init
This utility will walk you through creating a package.json file.
It only covers the most common items, and tries to guess sensible defaults.

See `npm help init` for definitive documentation on these fields
and exactly what they do.

Use `npm install <pkg>` afterwards to install a package and
save it as a dependency in the package.json file.

Press ^C at any time to quit.
package name: (demo)
version: (1.0.0)
description:
entry point: (index.js)
test command:
git repository:
keywords:
keywords:
keywords:
keywords:
author:
license: (ISC)
type: (commonjs)
About to write to C:\Users\sujit\Demo\package.json:

keywords:
author:
license: (ISC)
type: (commonjs)
About to write to C:\Users\sujit\Demo\package.json:

{
  "name": "demo",
  keywords:
  author:
  license: (ISC)
  type: (commonjs)
  About to write to C:\Users\sujit\Demo\package.json:

  license: (ISC)
  type: (commonjs)
  About to write to C:\Users\sujit\Demo\package.json:

  type: (commonjs)
  About to write to C:\Users\sujit\Demo\package.json:

```

**Conclusion-**

Setting up a Node.js project with an Express server for a Game store establishes the backbone for a scalable and interactive web application. By defining routes like Home, About, and Contact, the project achieves clear navigation and structure, ensuring a smooth user experience. The combination of Node.js's asynchronous efficiency and Express.js's simplicity makes this setup ideal for developing high-performance, modern web platforms.

## Experiment No-04

### Problem Statement-

To implement modular route handling and middleware logging for structured and traceable server operations.

### Theory-

In large-scale web applications, maintaining clean, readable, and scalable code is one of the most crucial requirements. As the number of routes in a Node.js + Express application grows, it becomes increasingly difficult to manage all routes in a single file. This is where the concept of **Modular Routing** and **Middleware Logging** comes into play. These techniques improve structure, maintainability, and traceability of the application.

### Modular Routing in Express.js

Routing refers to the mechanism that determines how an application responds to a client request for a specific endpoint, defined by a URL and HTTP method (GET, POST, PUT, DELETE). Initially, routes are written directly in the main `app.js` file, such as:

```
app.get('/', (req, res) => res.send('Home Page'));  
app.get('/about', (req, res) => res.send('About Page'));
```

While this approach works for small applications, it becomes impractical as the application scales. To solve this, **Express Router** provides a way to create modular, mountable route handlers. Developers can separate routes into multiple files, each representing a specific resource or functionality.

Example:

```
// userRoutes.js  
const express = require('express');  
const router = express.Router();  
  
router.get('/', (req, res) => res.send('User Home'));  
router.get('/profile', (req, res) => res.send('User Profile'));  
  
module.exports = router;
```

This route module is then imported into the main file:

```
const userRoutes = require('./routes/userRoutes');
app.use('/users', userRoutes);
```

This approach enhances **separation of concerns** — each module handles its own functionality independently. It also improves collaboration among team members, as developers can work on different route modules simultaneously.

## Middleware in Express

Middleware functions are one of the most powerful features of Express. They are functions that execute during the request–response cycle and have access to `req`, `res`, and the `next()` function. Middleware can perform operations such as authentication, error handling, input validation, or **logging** before the request reaches the final route handler.

Types of middleware include:

1. **Application-level middleware:** Applied globally to all routes using `app.use()`.
2. **Router-level middleware:** Applied only to specific route modules.
3. **Error-handling middleware:** Handles exceptions and unexpected server issues.

A simple example of a logging middleware:

```
app.use((req, res, next) => {
  console.log(`${req.method}    ${req.url}    at    ${new
Date().toISOString()}`);
  next();
});
```

This function logs the HTTP method, URL, and timestamp for every incoming request. Such logs help in performance monitoring, debugging, and security audits.

## Advantages of Modular Routing and Middleware

- Improved maintainability and readability.
- Enhanced scalability and reusability of code.
- Easier debugging and logging of request flow.

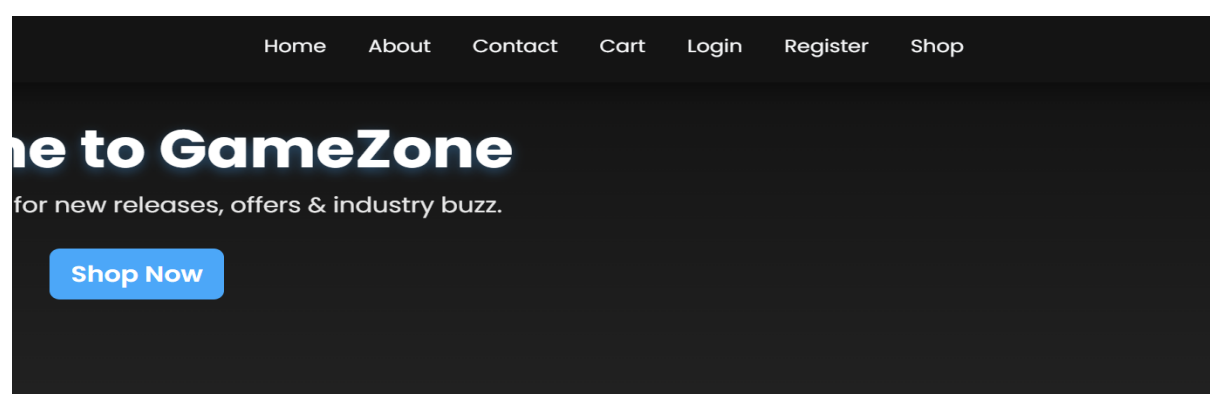
- Simplified team collaboration and version control.

In large enterprise systems, a combination of modular routing and middleware forms the backbone of microservices-oriented architectures.

## Code-

```
// =====  
  
// 🌐 MAIN ROUTES  
  
// =====  
  
app.get('/', (req, res) => res.render('index', { pageTitle: 'Home | GameZone' }));  
  
app.get('/about', (req, res) => res.render('about', { pageTitle: 'About | GameZone' }));  
  
app.get('/contact', (req, res) => res.render('contact', { pageTitle: 'Contact | GameZone' }));  
  
app.get('/cart', (req, res) => res.render('cart', { pageTitle: 'Cart | GameZone' }));  
  
app.get('/shop', (req, res) => res.render('shop', { pageTitle: 'Shop | GameZone' }));  
  
app.get('/login', (req, res) => res.render('login', { pageTitle: 'Login | GameZone' }));  
  
app.get('/register', (req, res) => res.render('register', { pageTitle: 'Register | GameZone' }));
```

## Output-



## Conclusion

Modular routing and middleware logging are fundamental to building professional-grade Express applications. They promote cleaner design, simplify debugging, and make large applications manageable. Logging middleware ensures full visibility into server activity, while modular routing provides a structured and maintainable codebase.

## EXPERIMENT 5: Create a Data Submission Form and Handle POST Requests

### Aim:

To build an HTML form and handle its POST request in Express to collect and display submitted data.

### Theory-

Forms are an essential way for users to interact with web applications — from submitting feedback and registering accounts to creating products or posting content. Understanding how forms work and how to handle **POST requests** is fundamental in server-side programming using Node.js and Express.

### Form Submission in Web Applications

In traditional web development, an HTML form is used to collect data from users. The form element defines an action (the endpoint to send data to) and a method (GET or POST). While GET requests append data to the URL, POST requests send it securely within the body of the HTTP request.

Example:

```
<form action="/feedback" method="POST">
  <label>Name:</label>
  <input type="text" name="username">
  <label>Message:</label>
  <textarea name="message"></textarea>
  <button type="submit">Submit</button>
</form>
```

When the user submits the form, the browser sends the data to the server.

### Handling POST Requests in Express

In Node.js with Express, we can handle form submissions using routes and middleware. Express automatically provides request parsing middleware for URL-encoded or JSON data.

```
app.use(express.urlencoded({ extended: true }));
app.use(express.json());
```

Then, we define a POST route:

```
app.post('/feedback', (req, res) => {  
  const { username, message } = req.body;  
  res.send(`Thank you ${username}, your feedback was:  
  ${message}`);  
});
```

Here, `req.body` contains the data submitted through the form. The data can be validated, sanitized, and stored in a database for further use.

## Dynamic Rendering after Submission

Server-side rendering using EJS or another templating engine allows displaying a confirmation page dynamically:

```
res.render('confirmation', { name: req.body.username, feedback:  
req.body.message });
```

The EJS file:

```
<h2>Feedback Confirmation</h2>  
<p>Thank you, <%= name %>! We received: "<%= feedback %>".</p>
```

## Security and Validation

When handling form data, developers must ensure:

- **Input validation** (checking data type, format, and length).
- **Input sanitization** (removing harmful characters).
- **CSRF protection** (Cross-Site Request Forgery prevention).
- **Error handling** for missing or invalid data.

## Importance of POST Handling



POST request handling forms the basis of almost every dynamic web application. Login forms, registration pages, comment systems, and dashboards all depend on it. Express simplifies this process, providing an elegant, modular way to manage input data securely.

## Code-

```
const mysql = require('mysql2');

const connection = mysql.createConnection({
  host: 'localhost',
  user: 'root',
  password: '', // keep empty unless set in XAMPP
  database: 'gamestore'
});

connection.connect((err) => {
  if (err) {
    console.error('✗ Database connection failed:', err.message);
  } else {
    console.log('☑ Connected to MySQL database');
  }
});

module.exports = connection;
```

## Conclusion

Data submission and POST handling are integral to interactive applications. Express simplifies these interactions, allowing developers to process and display user data dynamically. Proper validation and secure handling ensure both functionality and safety in real-world web systems.

## EXPERIMENT 6: Serve Dynamic Content Using Server-Side Rendering

### Aim:

To dynamically render pages using EJS templates populated with server-side data.

### Theory

In modern web development, websites are increasingly data-driven and dynamic. This means content displayed to the user changes depending on data, user actions, or context. One of the most efficient ways to achieve this in backend development is through **Server-Side Rendering (SSR)**. SSR is a technique where the server generates HTML content dynamically before sending it to the client's browser. In the Node.js ecosystem, this is often implemented using a templating engine like **EJS (Embedded JavaScript)**.

### Concept of Server-Side Rendering

Server-Side Rendering works by combining templates with data on the server to produce complete HTML pages. When a client sends a request (for example, to `/products`), the server fetches data — either from a database or a local source — and injects it into a view template. The resulting HTML is then sent to the browser. This process differs from **Client-Side Rendering (CSR)**, where the browser downloads JavaScript files and renders content dynamically after loading. SSR provides faster initial load times and is better for SEO since search engines can crawl the rendered HTML.

### EJS – Embedded JavaScript Templating

EJS is a lightweight templating language that lets you embed JavaScript logic directly within HTML. It supports features like looping, conditional statements, and partials. To set up EJS with Express:

```
app.set('view engine', 'ejs');
```

A simple route that renders a list of services:

```
app.get('/services', (req, res) => {  
  const services = [  
    { name: 'Web Design', price: 500 },  
    { name: 'Mobile Development', price: 800 },
```

```

    { name: 'SEO Optimization', price: 300 }
  ];

  res.render('services', { services });
});

```

The EJS file (`services.ejs`):

```

<h2>Our Services</h2>

<ul>

  <% services.forEach(service => { %>

    <li><%= service.name %> - $<%= service.price %></li>

  <% }); %>

</ul>

```

This will render an HTML list dynamically populated with data.

### Advantages of SSR

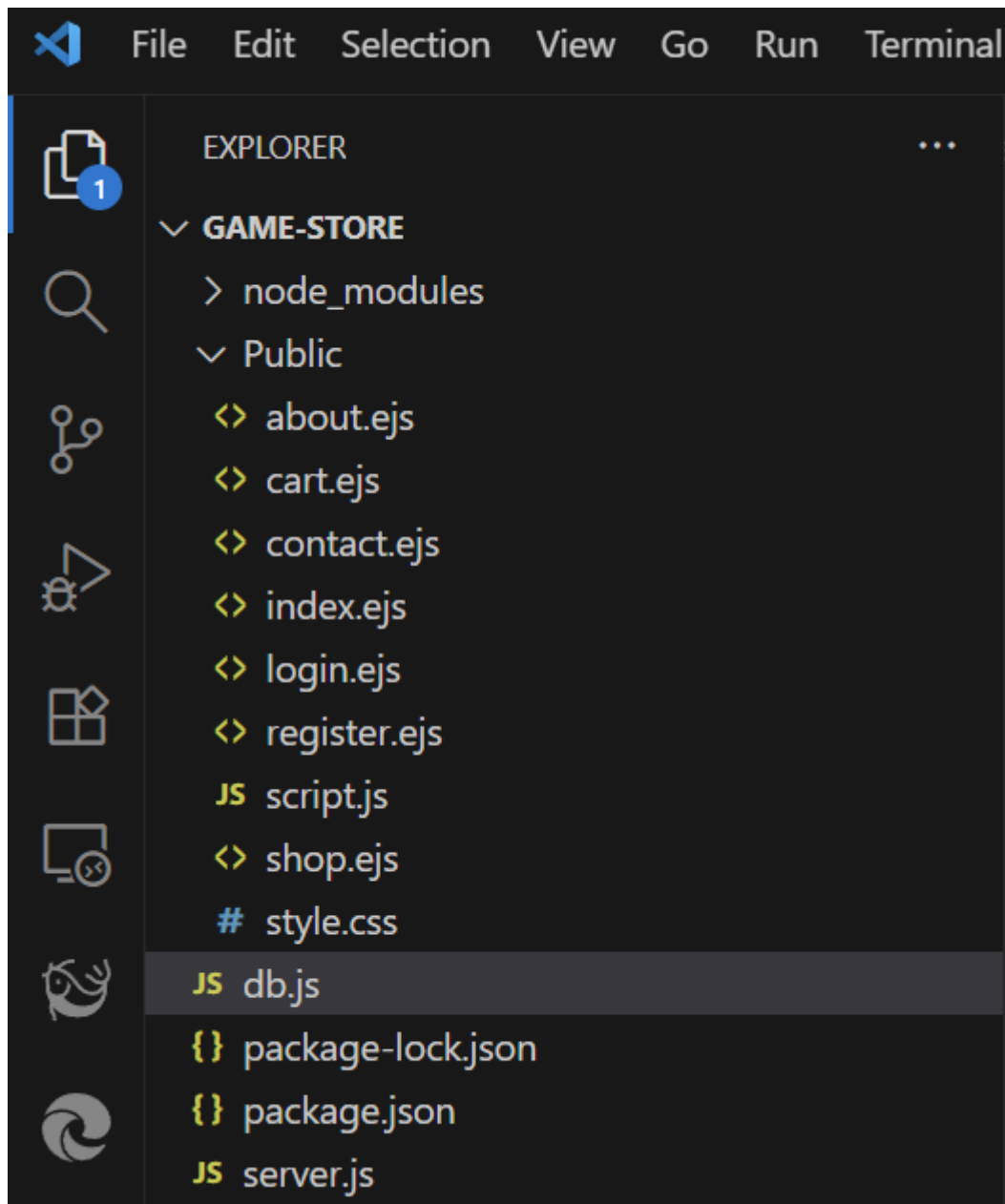
1. **Improved SEO:** Since content is rendered before reaching the browser, search engines can index pages easily.
2. **Faster first paint:** The user sees meaningful content quicker compared to client-rendered pages.
3. **Reduced JavaScript dependency:** The server handles most rendering logic.
4. **Better performance on low-end devices:** Less computation occurs on the client side.

### Use Cases

- E-commerce sites displaying dynamic product listings.

- Blogs fetching posts from a database.
- Dashboards showing live updates.

### Code-



```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8" />
```

```
<meta name="viewport" content="width=device-width, initial-scale=1.0"
/>

<title><%= pageTitle %></title>

<!-- Bootstrap -->

<link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.m
in.css" rel="stylesheet" />

<!-- AOS Animations -->

<link href="https://unpkg.com/aos@2.3.1/dist/aos.css"
rel="stylesheet">

<!-- Google Font -->

<link
href="https://fonts.googleapis.com/css2?family=Poppins:wght@400;600;700
&display=swap" rel="stylesheet" />

<style>

  body {

    font-family: 'Poppins', sans-serif;

    background-color: #0e0e0e;

    color: #f8f9fa;

    overflow-x: hidden;

    scroll-behavior: smooth;

  }
```

```
/* Navbar */

.navbar {

  background: rgba(20, 20, 20, 0.95);

  backdrop-filter: blur(10px);

  box-shadow: 0 4px 25px rgba(0, 0, 0, 0.5);

  transition: background 0.4s ease;

}

.navbar-brand {

  font-size: 1.6rem;

  color: #00aaff !important;

  font-weight: 700;

}

.nav-link {

  color: #f8f9fa !important;

  margin: 0 8px;

  font-weight: 500;

  transition: color 0.3s ease;

}

.nav-link:hover {

  color: #00aaff !important;

}

/* Hero Section */

header {

  background: linear-gradient(180deg, #111, #222);
```

```
padding: 6rem 1rem;

text-align: center;

color: #fff;

}

header h1 {

    font-weight: 700;

    font-size: 3rem;

    text-shadow: 0 3px 10px rgba(0, 170, 255, 0.4);

}

header p {

    font-size: 1.2rem;

    margin-bottom: 1.5rem;

    opacity: 0.9;

}

header .btn {

    background-color: #00aaff;

    border: none;

    font-weight: 600;

    transition: transform 0.3s ease, background 0.3s ease;

}

header .btn:hover {

    transform: scale(1.1);

    background-color: #007acc;

}
```



```
/* Section Titles */

section h2 {

    color: #00aaff;

    font-weight: 700;

    margin-bottom: 2rem;

    text-align: center;

}


/* Game Cards */

.game-card {

    background: #1b1b1b;

    border: none;

    border-radius: 14px;

    transition: transform 0.4s ease, box-shadow 0.4s ease;

    overflow: hidden;

}

.game-card:hover {

    transform: translateY(-10px);

    box-shadow: 0 10px 35px rgba(0, 170, 255, 0.4);

}

.game-card img {

    border-radius: 14px 14px 0 0;

}


/* News Cards */
```

```
.news-card {  
  
    background: #1a1a1a;  
  
    border-radius: 12px;  
  
    padding: 1.5rem;  
  
    height: 100%;  
  
    transition: transform 0.4s ease, background 0.3s ease;  
  
}  
  
.news-card:hover {  
  
    transform: translateY(-8px);  
  
    background: #232323;  
  
}  
  
.news-card h4 {  
  
    color: #00aaff;  
  
    font-weight: 600;  
  
    margin-bottom: 0.6rem;  
  
}  
  
/* Stats */  
  
.stats {  
  
    background: linear-gradient(90deg, #0e0e0e, #1c1c1c);  
  
    padding: 4rem 1rem;  
  
    border-top: 1px solid #222;  
  
    border-bottom: 1px solid #222;  
  
}
```

```
.stats h3 {

    font-size: 2.4rem;

    color: #00aaff;

}

.stats p {

    color: #bbb;

    font-size: 1rem;

}

/* Footer */

footer {

    background: #1b1b1b;

    color: #bbb;

    padding: 1.5rem 0;

    text-align: center;

    font-size: 0.9rem;

}

/* Scroll animation delay for cards */

[data-aos] {

    transition-property: transform, opacity;

    transition-duration: 1s;

}

/* Responsive Adjustments */
```

```

@media (max-width: 992px) {

    header h1 { font-size: 2.2rem; }

    header p { font-size: 1rem; }

}

@media (max-width: 576px) {

    header { padding: 3rem 1rem; }

    header h1 { font-size: 1.8rem; }

}

</style>
</head>
<body>

<!-- Navbar -->

<nav class="navbar navbar-expand-lg navbar-dark fixed-top">

    <div class="container">

        <a class="navbar-brand" href="/">🎮 GameZone</a>

        <button class="navbar-toggler" type="button" data-bs-
toggle="collapse" data-bs-target="#navbarNav">

            <span class="navbar-toggler-icon"></span>

        </button>

        <div class="collapse navbar-collapse justify-content-end"
id="navbarNav">

            <ul class="navbar-nav align-items-center">

                <li class="nav-item"><a class="nav-link active"
href="/">Home</a></li>

```

```
        <li class="nav-item"><a class="nav-link"
href="/about">About</a></li>

        <li class="nav-item"><a class="nav-link"
href="/contact">Contact</a></li>

        <li class="nav-item"><a class="nav-link"
href="/cart">Cart</a></li>

        <li class="nav-item"><a class="nav-link"
href="/login">Login</a></li>

        <li class="nav-item"><a class="nav-link"
href="/register">Register</a></li>

        <li class="nav-item"><a class="nav-link"
href="/shop">Shop</a></li>

    </ul>

</div>

</div>

</nav>

<!-- Hero Section -->

<header data-aos="fade-down">

    <div class="container">

        <h1>Welcome to GameZone</h1>

        <p>Your one-stop hub for new releases, offers & industry
buzz.</p>

        <a href="/shop" class="btn btn-lg text-white">Shop Now</a>

    </div>

</header>

<!-- Featured Games -->
```

```

<section class="container py-5">

  <h2 data-aos="zoom-in">🔥 Featured Games</h2>

  <div class="row g-4 justify-content-center">

    <div class="col-md-4 col-sm-6" data-aos="fade-up">

      <div class="card game-card">

        <div class="card-body text-center">

          <h5>Cyberpunk 2077</h5>

          <p>$59.99</p>

          <a href="/cart" class="btn btn-primary">Add to Cart</a>

        </div>

      </div>

    </div>

    <div class="col-md-4 col-sm-6" data-aos="fade-up" data-aos-
delay="100">

      <div class="card game-card">

        <div class="card-body text-center">

          <h5>Elden Ring</h5>

          <p>$69.99</p>

          <a href="/cart" class="btn btn-primary">Add to Cart</a>

        </div>

      </div>

    </div>
  </div>

```

```

</div>

<div class="col-md-4 col-sm-6" data-aos="fade-up" data-aos-
delay="200">

  <div class="card game-card">

    <div class="card-body text-center">

      <h5>God of War: Ragnarok</h5>

      <p>$49.99</p>

      <a href="/cart" class="btn btn-primary">Add to Cart</a>

    </div>

  </div>

</div>

</div>

</div>

</section>

<!-- News -->

<section class="container py-5">

  <h2 data-aos="zoom-in"><img alt="Gaming icon" data-bbox="408 681 428 693"/> Gaming News</h2>

  <div class="row g-4">

    <div class="col-lg-4 col-md-6" data-aos="fade-right">

      <div class="news-card">

        <h4><img alt="GTA icon" data-bbox="271 818 291 831"/> GTA VI Trailer Drops Soon</h4>

        <p>Rockstar confirms the official GTA VI trailer release.
Expect cutting-edge visuals and insane realism!</p>

```

```

    </div>

</div>

<div class="col-lg-4 col-md-6" data-aos="fade-up">

    <div class="news-card">

        <h4><img alt="trophy icon" data-bbox="271 223 291 238"/> Esports World Cup 2025</h4>

        <p>Get ready for the ultimate showdown – top teams compete
across Valorant, Fortnite, and Apex Legends.</p>

    </div>

</div>

<div class="col-lg-4 col-md-6" data-aos="fade-left">

    <div class="news-card">

        <h4><img alt="twitch icon" data-bbox="271 445 291 460"/> Streamer Spotlight: Valkyrae</h4>

        <p>Valkyrae's charity stream shatters records, proving gaming
is more than competition – it's community.</p>

    </div>

</div>

</div>

</section>

<!-- Stats -->

<section class="stats text-center" data-aos="fade-up">

    <div class="container">

        <h2><img alt="bar chart icon" data-bbox="228 801 248 816"/> Industry Highlights</h2>

        <div class="row g-4">

            <div class="col-md-3 col-6"><h3>$200B+</h3><p>Revenue</p></div>

```



```
<div class="col-md-3 col-6"><h3>3.2B+</h3><p>Gamers</p></div>

<div class="col-md-3 col-6"><h3>10K+</h3><p>Games in
2025</p></div>

<div class="col-md-3 col-6"><h3>95%</h3><p>Growth
Rate</p></div>

</div>

</div>

</section>

<!-- Footer -->

<footer>

  <p>&copy; 2025 GameZone. All rights reserved.</p>

</footer>

<!-- Scripts -->

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>

<script src="https://unpkg.com/aos@2.3.1/dist/aos.js"></script>

<script>

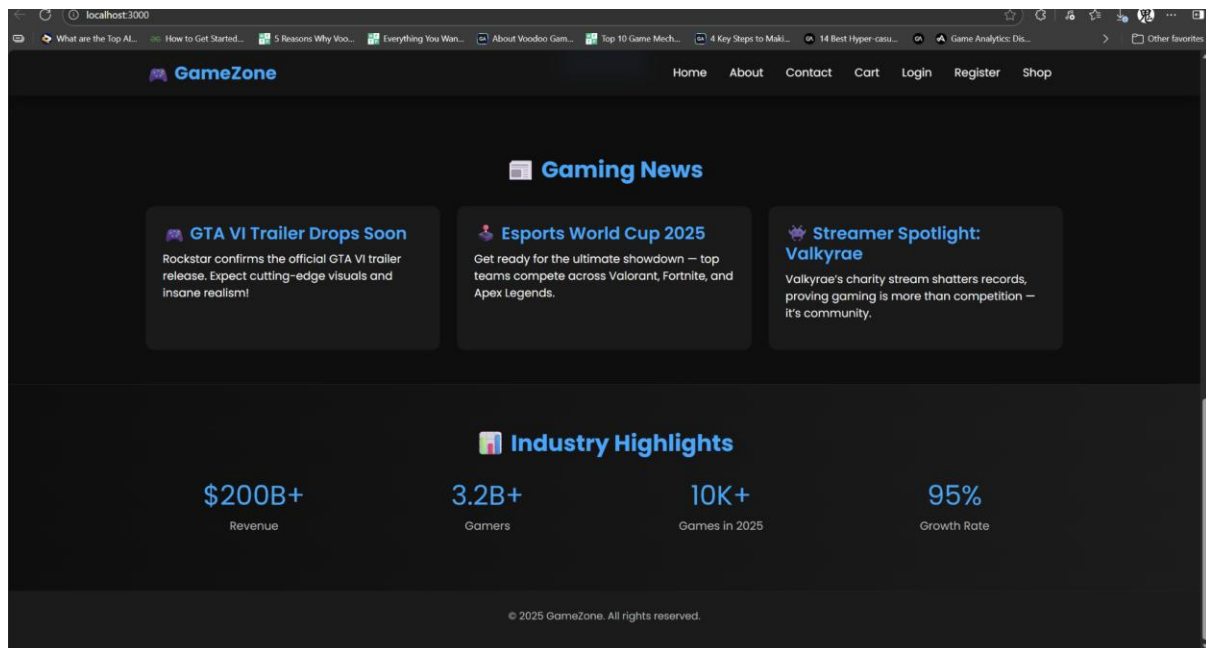
  AOS.init({ duration: 1000, once: true });

</script>

</body>

</html>
```

## Output-



## Conclusion

Server-Side Rendering using EJS provides an efficient method for creating fast, SEO-friendly, and dynamic websites. It bridges the gap between traditional static websites and modern interactive applications, offering the benefits of speed and maintainability.

## 7. Create Dynamic Detail Pages Using URL Parameters

### Aim:

To generate dynamic pages that display item details using Express route parameters and EJS rendering.

### Theory

In dynamic web applications, users often need to view details about specific items — like a product, article, or user profile. Rather than creating separate pages for each item, **URL parameters** allow developers to build a single, flexible route that dynamically generates content based on the provided parameter. In Express.js, this is achieved using **route parameters** (denoted by a colon `:` in the URL).

### Understanding Route Parameters

A route parameter is a variable part of the URL that can capture values passed by the client. For instance, `/product/:id` defines a route pattern where `:id` is a placeholder for a dynamic value (e.g., `/product/2`). When a request is made, Express captures this value and makes it available under `req.params`.

Example:

```
app.get('/product/:id', (req, res) => {  
  const products = [  
    { id: 1, name: 'Gaming Laptop', price: 90000 },  
    { id: 2, name: 'Wireless Mouse', price: 1500 },  
  ];  
  
  const product = products.find(p => p.id == req.params.id);  
  
  res.render('productDetail', { product });  
});
```

EJS template:

```
<h2><%= product.name %></h2>
```

```
<p>Price: ₹<%= product.price %></p>
```

When the user navigates to `/product/1`, it will display details of the first product. This approach eliminates redundancy and enables a scalable application structure.

## Chaining Multiple Parameters

Express supports multiple route parameters:

```
app.get('/category/:catId/product/:prodId', (req, res) => {
  res.send(`Category:      ${req.params.catId},      Product:
${req.params.prodId}`);
});
```

This feature is especially useful in e-commerce or content management systems.

## Benefits

- **Scalability:** One template serves hundreds of pages dynamically.
- **Code reusability:** Reduces repetition of similar routes.
- **Improved UX:** Allows meaningful, readable URLs for users.
- **SEO optimization:** Search engines prefer structured URLs with identifiers or names.

## Code-

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8" />

  <meta name="viewport" content="width=device-width, initial-scale=1.0"
/>
```

```
<title>GameZone - Shop</title>

<!-- Bootstrap CSS -->

<link

href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.m
in.css"

rel="stylesheet"

/>

<!-- Google Font -->

<link
href="https://fonts.googleapis.com/css2?family=Poppins:wght@400;600;700
&display=swap" rel="stylesheet">

<style>

* {

    font-family: 'Poppins', sans-serif;

    scroll-behavior: smooth;

}

body {

    background: linear-gradient(135deg, #0f0f1a, #1a1a2e);

    color: #fff;

    overflow-x: hidden;

}
```

```
/* Navbar */

.navbar {

    background: rgba(15, 15, 25, 0.95);

    backdrop-filter: blur(10px);

    box-shadow: 0 2px 15px rgba(0, 0, 0, 0.5);

    transition: all 0.3s ease;

}

.navbar a.nav-link {

    color: #aaa !important;

    margin-right: 15px;

    transition: 0.3s;

}

.navbar a.nav-link:hover,

.navbar a.nav-link.active {

    color: #00bfff !important;

    text-shadow: 0 0 8px #00bfff;

}


/* Hero Section */

header {

    background: linear-gradient(135deg, #141420, #1f1f3b);

    padding: 100px 0;

    color: #fff;

    position: relative;

}
```

```
header::after {  
  
    content: "";  
  
    position: absolute;  
  
    bottom: 0;  
  
    left: 0;  
  
    height: 5px;  
  
    width: 100%;  
  
    background: linear-gradient(90deg, #00bfff, #ff0080);  
  
}  
  
header h1 {  
  
    font-size: 3rem;  
  
    text-shadow: 0 0 15px rgba(0, 191, 255, 0.7);  
  
}  
  
header p {  
  
    color: #ccc;  
  
    font-size: 1.2rem;  
  
}  
  
/* Game Cards */  
  
.game-card {  
  
    background: rgba(255, 255, 255, 0.05);  
  
    border: none;  
  
    border-radius: 20px;  
  
    overflow: hidden;  
  
    transition: all 0.4s ease;
```

```
    cursor: pointer;

    position: relative;
}

.game-card:hover {

    transform: translateY(-10px) scale(1.03);

    box-shadow: 0 0 30px rgba(0, 191, 255, 0.3);
}

.game-card img {

    border-radius: 15px 15px 0 0;

    transition: 0.4s ease;
}

.game-card:hover img {

    filter: brightness(1.2);

    transform: scale(1.05);
}

.card-body h5 {

    font-weight: 600;

    color: #00bfff;
}

.btn-primary {
```



```
background: linear-gradient(90deg, #00bfff, #ff0080);

border: none;

font-weight: 600;

transition: all 0.3s ease;

border-radius: 50px;
}

.btn-primary:hover {

transform: scale(1.05);

box-shadow: 0 0 20px #00bfff;
}

/* Footer */

footer {

background: rgba(15, 15, 25, 0.95);

color: #aaa;

padding: 20px 0;

margin-top: 50px;
}

footer p {

margin: 0;

font-size: 0.9rem;
}
```

```
/* Responsive */

@media (max-width: 768px) {

  header h1 {

    font-size: 2.2rem;

  }

  header p {

    font-size: 1rem;

  }

}

</style>

</head>

<body>

  <!-- Navbar -->

  <nav class="navbar navbar-expand-lg navbar-dark fixed-top">

    <div class="container">

      <a class="navbar-brand fw-bold text-light" href="/">🎮
GameZone</a>

      <button

        class="navbar-toggler"

        type="button"

        data-bs-toggle="collapse"

        data-bs-target="#navbarNav"

        aria-controls="navbarNav"

        aria-expanded="false"

        >
```

```

        <span class="navbar-toggler-icon"></span>

    </button>

    <div class="collapse navbar-collapse" id="navbarNav">

        <ul class="navbar-nav ms-auto text-center">

            <li class="nav-item"><a class="nav-link"
href="/">Home</a></li>

            <li class="nav-item"><a class="nav-link active"
href="/shop.html">Shop</a></li>

            <li class="nav-item"><a class="nav-link"
href="/offers">Offers</a></li>

            <li class="nav-item"><a class="nav-link"
href="/contact">Contact</a></li>

            <li class="nav-item"><a class="nav-link" href="/cart">🛒
Cart</a></li>

        </ul>

    </div>

</div>

</nav>

<!-- Hero Section -->

<header class="text-center">

    <div class="container mt-5 pt-4">

        <h1 class="fw-bold mb-3">Shop the Best Games</h1>

        <p class="lead mb-0">Find your next adventure – from RPGs to
Racing!</p>

    </div>

</header>

```

```

<!-- Shop Section -->

<section class="container py-5">

  <h2 class="text-center mb-5 fw-bold text-uppercase text-info">🛒
Game Store</h2>

  <div class="row g-4">

    <!-- Game Cards -->

    <div class="col-md-4">

      <div class="card game-card">

        <div class="card-body text-center">

          <h5>Cyberpunk 2077</h5>

          <p class="text-light">$59.99</p>

          <a href="/cart" class="btn btn-primary">Add to Cart</a>

        </div>

      </div>

    </div>

    <div class="col-md-4">

      <div class="card game-card">

        <div class="card-body text-center">

          <h5>Elden Ring</h5>

```

```

        <p class="text-light">$69.99</p>

        <a href="/cart" class="btn btn-primary">Add to Cart</a>

    </div>

</div>

</div>

<div class="col-md-4">

    <div class="card game-card">

        <div class="card-body text-center">

            <h5>God of War Ragnarök</h5>

            <p class="text-light">$49.99</p>

            <a href="/cart" class="btn btn-primary">Add to Cart</a>

        </div>

    </div>

</div>

<div class="col-md-4">

    <div class="card game-card">

        <div class="card-body text-center">

            <h5>The Last of Us Part I</h5>

            <p class="text-light">$39.99</p>

```

```

        <a href="/cart" class="btn btn-primary">Add to Cart</a>

    </div>

</div>

</div>

<div class="col-md-4">

    <div class="card game-card">

        <div class="card-body text-center">

            <h5>Horizon Forbidden West</h5>

            <p class="text-light">$44.99</p>

            <a href="/cart" class="btn btn-primary">Add to Cart</a>

        </div>

    </div>

</div>

<div class="col-md-4">

    <div class="card game-card">

        <div class="card-body text-center">

            <h5>Red Dead Redemption 2</h5>

            <p class="text-light">$29.99</p>

            <a href="/cart" class="btn btn-primary">Add to Cart</a>

```

```
        </div>

    </div>

</div>

</div>

</section>

<!-- Footer -->

<footer class="text-center">

    <p>&copy; 2025 GameZone. Built for Gamers ❤️</p>

</footer>

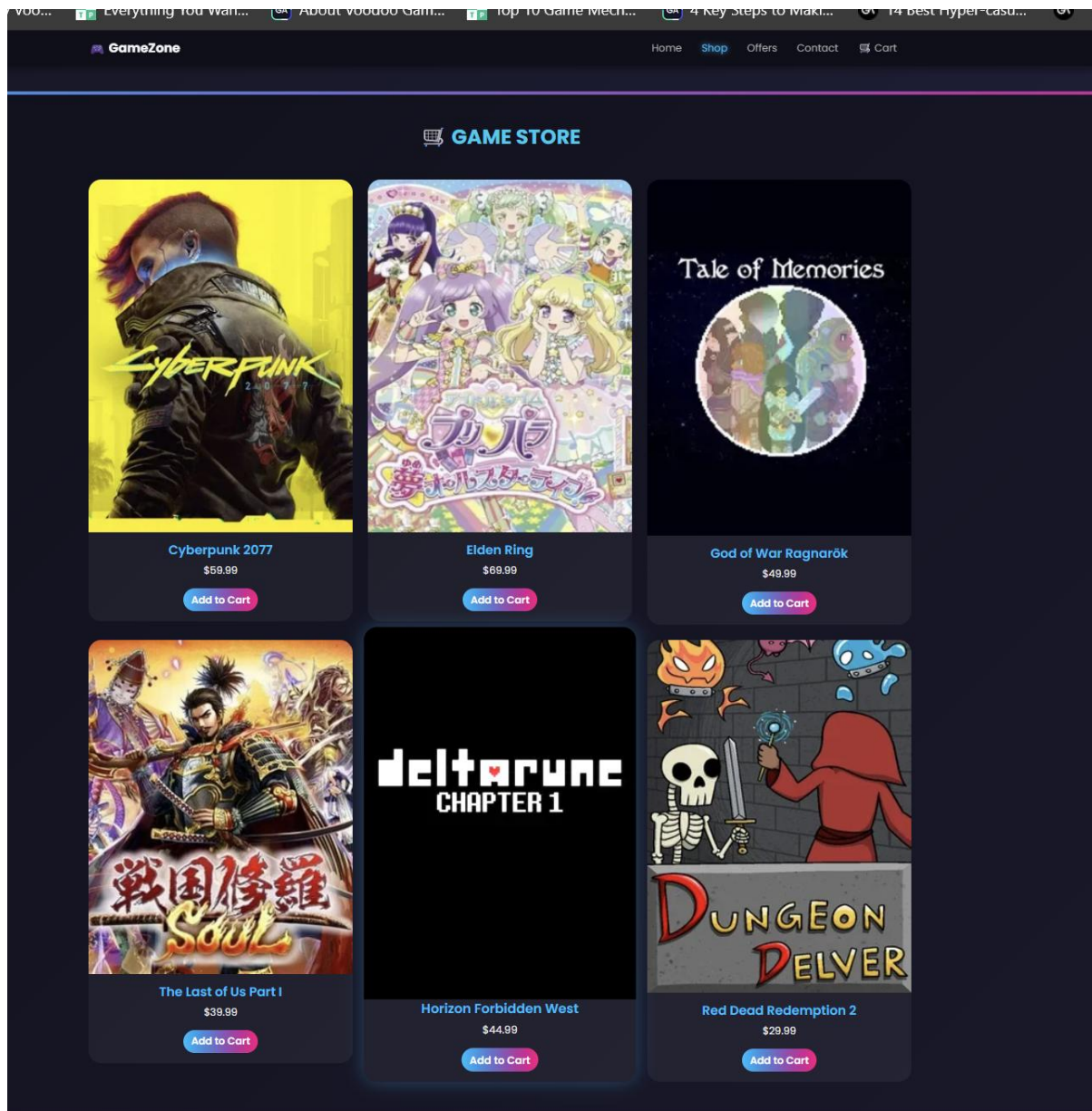
<!-- Bootstrap JS -->

<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>

</body>

</html>
```

## Output-



## Conclusion

Using route parameters enables the creation of flexible, scalable, and data-driven web applications. Developers can dynamically render specific pages without duplicating code, improving both maintainability and performance.



## 8. Build a Reusable Layout Using EJS Partial

### Aim:

To maintain UI consistency by creating reusable EJS partials for common components like header, footer, and navbar.

### Theory

Consistency is vital in web design. Every page should share common elements like headers, footers, and navigation menus. Writing these components repeatedly across pages is inefficient. To solve this, templating engines like EJS introduce **partials** — reusable HTML snippets included in multiple templates.

### Concept of Partial

Partials are fragments of HTML and EJS that can be embedded into other templates. For instance, a navigation bar can be written once as `navbar.ejs` and then included in every page using:

```
<%- include('partials/navbar') %>
```

This not only ensures uniformity across all pages but also reduces maintenance time. A change in the partial file automatically reflects across all pages that include it.

Example folder structure:

```
views/  
├── partials/  
│   ├── header.ejs  
│   ├── footer.ejs  
├── home.ejs  
└── about.ejs
```

### header.ejs

```
<header class="bg-dark text-white p-3">
```

```
<h1>My Website</h1>

<nav><a href="/">Home</a> | <a href="/about">About</a></nav>

</header>
```

### home.ejs

```
<%- include('partials/header') %>

<main>

  <h2>Welcome to the homepage!</h2>

</main>

<%- include('partials/footer') %>
```

### Advantages

1. **Consistency:** All pages share the same UI components.
2. **Maintainability:** Update one file to modify all pages.
3. **Reusability:** Components can be reused in multiple projects.
4. **Reduced redundancy:** Less repetitive HTML code.

### Conclusion

Using EJS partials creates clean, maintainable, and scalable UI architecture. It ensures a professional look, reduces development time, and simplifies large multi-page application management.

## 9. Connect Node.js to MySQL and Perform Basic CRUD

### Aim:

To integrate Node.js with MySQL and perform CRUD (Create, Read, Update, Delete) operations.

### Theory

For web applications to be practical, they must store and retrieve data persistently. While in-memory data disappears after server restart, databases like **MySQL** provide structured and durable data storage. Node.js interacts with MySQL using the **mysql2** or **sequelize** package, allowing the execution of **CRUD operations** (Create, Read, Update, Delete).

### Connecting Node.js and MySQL

To connect:

```
const mysql = require('mysql2');

const db = mysql.createConnection({
  host: 'localhost',
  user: 'root',
  password: '',
  database: 'shopdb'
});

db.connect(err => {
  if (err) throw err;
  console.log('Connected to MySQL');
});
```

### CRUD Operations

### 1. Create (INSERT):

```
db.query('INSERT INTO products (name, price) VALUES (?, ?)',  
[ 'Mouse', 500]);
```

### 2. Read (SELECT):

```
db.query('SELECT * FROM products', (err, results) =>  
console.log(results));
```

### 3. Update:

```
db.query('UPDATE products SET price=? WHERE id=?', [600, 1]);
```

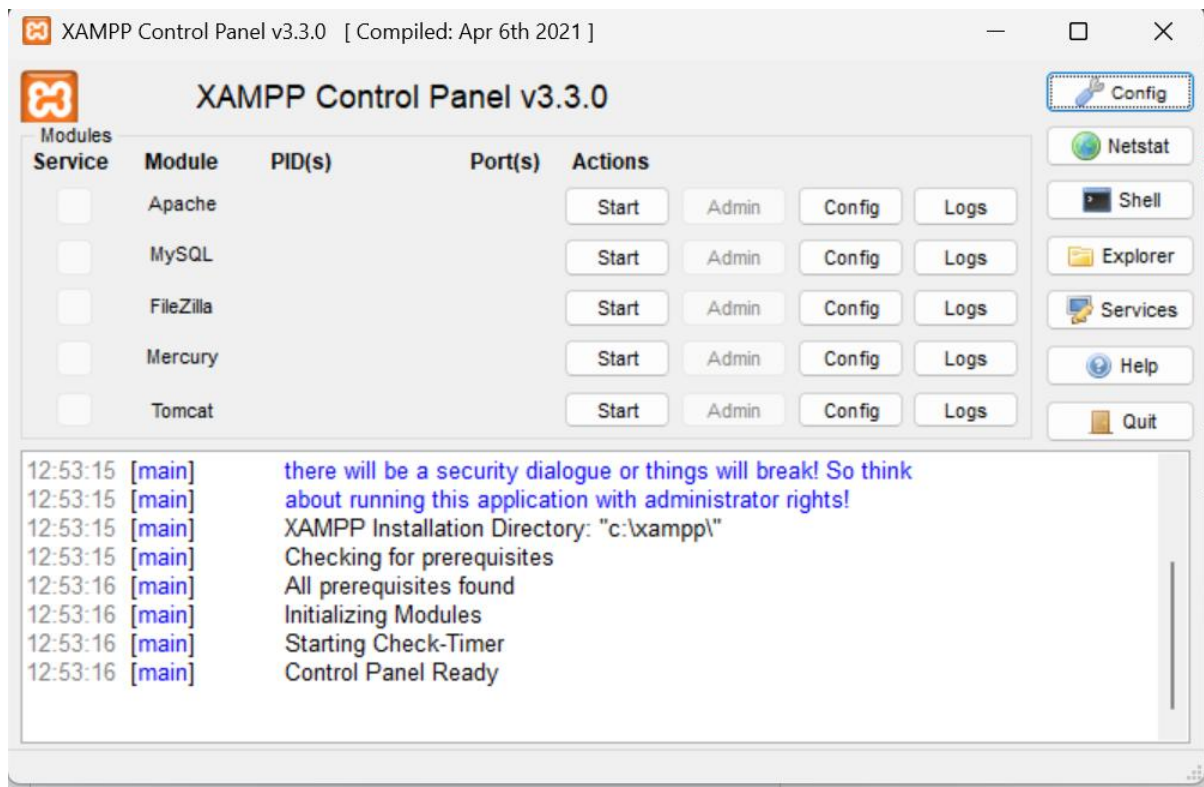
### 4. Delete:

```
db.query('DELETE FROM products WHERE id=?', [1]);
```

## Advantages

- MySQL provides relational data management using tables, primary keys, and relationships.
- Node.js allows asynchronous operations, improving performance for concurrent queries.
- CRUD operations form the backbone of all dynamic data applications.

## Code-



```
const express = require('express');

const path = require('path');

const bodyParser = require('body-parser');

const connection = require('./db'); // Ensure db.js exports a
MySQL/MariaDB connection

const app = express();

// =====

// ⚙ VIEW & STATIC SETUP

// =====

app.set('views', path.join(__dirname, 'public'));

app.set('view engine', 'ejs');

app.use(express.static(path.join(__dirname, 'public')));
```

```

app.use(bodyParser.urlencoded({ extended: true }));

app.use(bodyParser.json());

// =====

// 🌐 MAIN ROUTES

// =====

app.get('/', (req, res) => res.render('index', { pageTitle: 'Home |
GameZone' }));

app.get('/about', (req, res) => res.render('about', { pageTitle: 'About
| GameZone' }));

app.get('/contact', (req, res) => res.render('contact', { pageTitle:
'Contact | GameZone' }));

app.get('/cart', (req, res) => res.render('cart', { pageTitle: 'Cart |
GameZone' }));

app.get('/shop', (req, res) => res.render('shop', { pageTitle: 'Shop |
GameZone' }));

app.get('/login', (req, res) => res.render('login', { pageTitle: 'Login
| GameZone' }));

app.get('/register', (req, res) => res.render('register', { pageTitle:
'Register | GameZone' }));

// =====

// 📝 REGISTER ROUTE

// =====

app.post('/register', (req, res) => {

  const { username, email, phone, dob, genre, password } = req.body;

  if (!username || !email || !phone || !dob || !genre || !password) {

```

```

    return res.status(400).json({ message: '⚠ All fields are
required.' });

}

const sql = 'INSERT INTO users (username, email, phone, dob, genre,
password) VALUES (?, ?, ?, ?, ?, ?)';

connection.query(sql, [username, email, phone, dob, genre, password],
(err, result) => {

    if (err) {

        console.error('✗ Error inserting data:', err);

        return res.status(500).json({ message: 'Database error. Please
try again.' });

    }

    console.log('☑ User registered successfully:', result.insertId);

    res.json({ message: 'User registered successfully!' });

});

});

// =====

// 🔑 LOGIN ROUTE

// =====

app.post('/login', (req, res) => {

    const { email, password } = req.body;

    if (!email || !password) {

```

```

    return res.status(400).json({ message: '⚠ Email and password are
required.' });

}

const sql = 'SELECT * FROM users WHERE email = ? AND password = ?';

connection.query(sql, [email, password], (err, results) => {

    if (err) {

        console.error('❌ Error checking login:', err);

        return res.status(500).json({ message: 'Server error. Please try
again.' });

    }

    if (results.length > 0) {

        console.log('✅ ${email} logged in successfully!');

        res.json({ message: 'Login successful!' });

    } else {

        res.json({ message: '❌ Invalid email or password!' });

    }

});

});

// =====

// 🚀 START SERVER

// =====

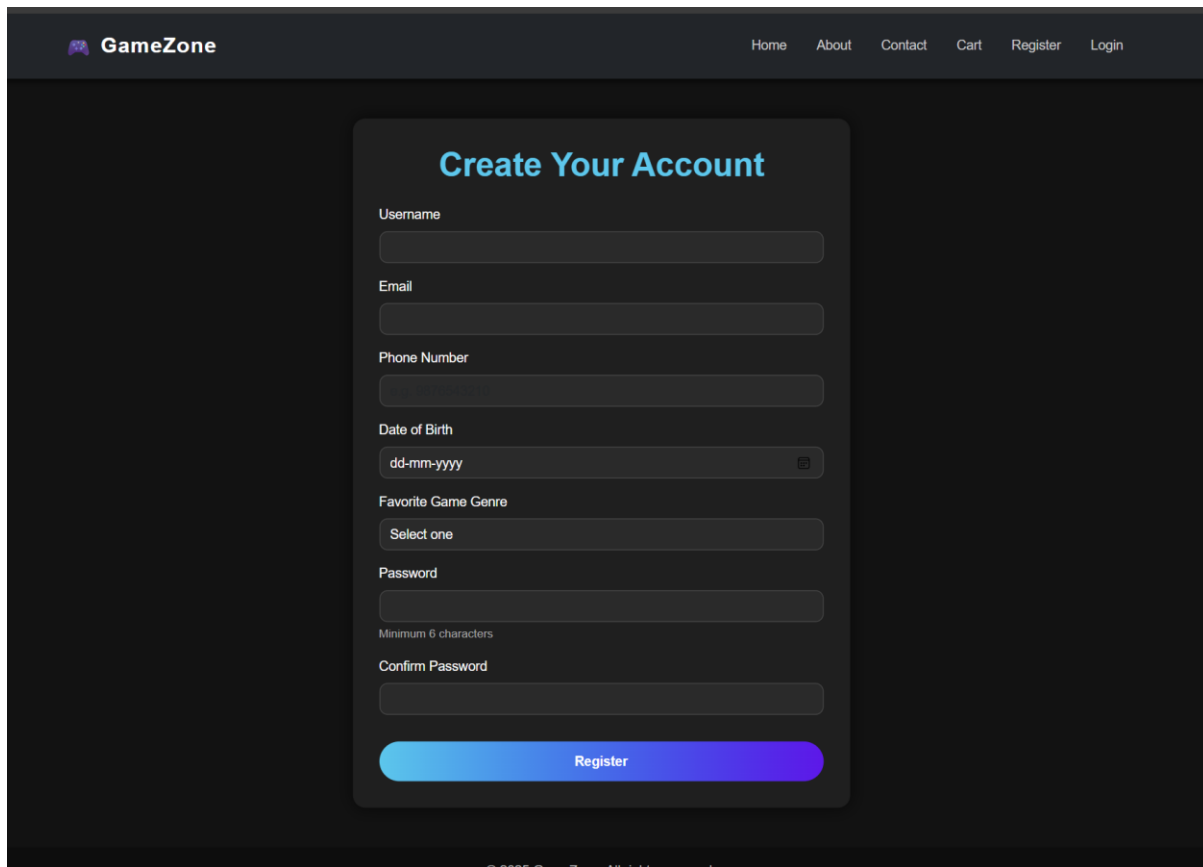
const PORT = process.env.PORT || 3000;

```



```
app.listen(PORT, () => console.log(`🎮 GameZone server running at  
http://localhost:${PORT}`));
```

## Output-



The screenshot displays the GameZone web application interface. At the top, a dark navigation bar contains the GameZone logo on the left and links for Home, About, Contact, Cart, Register, and Login on the right. The main content area features a central registration form titled "Create Your Account" in a light blue font. The form includes several input fields: Username, Email, Phone Number, Date of Birth (with a calendar icon), Favorite Game Genre (a dropdown menu showing "Select one"), Password, and Confirm Password. A note "Minimum 6 characters" is visible below the Password field. At the bottom of the form is a prominent blue "Register" button. The footer of the page shows the copyright notice "© 2025 GameZone. All rights reserved."

## Conclusion

Integrating Node.js with MySQL allows efficient data handling and persistence. CRUD operations enable complete database interactivity, making the web application functional and real-world ready.

## 10. Develop RESTful APIs for Data Access and Manipulation

### Aim:

To build and test RESTful APIs for performing CRUD operations using [Express.js](#).

### Theory

A **RESTful API (Representational State Transfer)** is a standardized way for systems to communicate over HTTP. In a REST API, each resource (like users or products) is identified by a unique URL, and different HTTP methods define the type of operation.

### REST Architecture

REST uses:

- **GET** → Read data
- **POST** → Create data
- **PUT/PATCH** → Update data
- **DELETE** → Remove data

Example Express routes:

```
app.get('/api/products', (req, res) => res.json(products));

app.post('/api/products', (req, res) => {
  products.push(req.body); res.json({ message: 'Added' }); });

app.put('/api/products/:id', (req, res) => res.json({ message:
'Updated' }));

app.delete('/api/products/:id', (req, res) => res.json({
message: 'Deleted' }));
```

### Testing with Postman

Postman allows developers to test API endpoints easily by sending requests and verifying JSON responses, ensuring reliability before integration with front-end apps.

## Best Practices

- Use meaningful routes (`/api/users/:id` instead of `/getUserData`).
- Return consistent JSON responses with status codes.
- Implement error handling and input validation.
- Use middleware for logging and authentication.

## Advantages

- Platform independence — APIs work across mobile, web, and desktop.
- Scalable and modular architecture.
- Facilitates separation between frontend and backend.

## Code-

```
<!DOCTYPE html>

<html lang="en">

<head>

  <meta charset="UTF-8" />

  <meta name="viewport" content="width=device-width, initial-scale=1.0" />

  <title>Register - GameZone</title>

  <!-- Bootstrap & Animations -->

  <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css" rel="stylesheet" />

  <link
href="https://cdnjs.cloudflare.com/ajax/libs/animate.css/4.1.1/animate.min.css" rel="stylesheet" />
```

```
<style>

  body {

    background-color: #121212;

    color: #fff;

    font-family: "Poppins", sans-serif;

  }


  .navbar {

    box-shadow: 0 4px 8px rgba(0, 0, 0, 0.5);

  }


  .navbar-brand {

    font-weight: 700;

    font-size: 1.6rem;

    letter-spacing: 1px;

  }


  .nav-link {

    color: #ccc !important;

    margin: 0 10px;

    transition: all 0.3s ease;

  }


  .nav-link:hover {
```

```
    color: #0dcaf0 !important;

    transform: scale(1.05);

}

main {

    max-width: 600px;

    margin: 3rem auto;

    background-color: #1f1f1f;

    border-radius: 15px;

    padding: 2rem;

    box-shadow: 0 0 20px rgba(0, 0, 0, 0.6);

    animation: fadeInUp 1s ease;

}

h1 {

    text-align: center;

    margin-bottom: 1.5rem;

    font-weight: 700;

    color: #0dcaf0;

}

label {

    font-weight: 500;

    margin-bottom: 5px;

}
```

```
.form-control {  
  
  background-color: #2a2a2a;  
  
  border: 1px solid #444;  
  
  color: #fff;  
  
  border-radius: 10px;  
  
}
```

```
.form-control:focus {  
  
  background-color: #333;  
  
  border-color: #0dcaf0;  
  
  box-shadow: 0 0 8px #0dcaf0;  
  
}
```

```
.btn-primary {  
  
  background: linear-gradient(90deg, #0dcaf0, #6610f2);  
  
  border: none;  
  
  width: 100%;  
  
  padding: 12px;  
  
  border-radius: 30px;  
  
  font-weight: bold;  
  
  transition: 0.3s;  
  
}
```

```
.btn-primary:hover {
```

```

        transform: scale(1.05);

        opacity: 0.9;
    }

    footer {

        background: #0d0d0d;

        padding: 1rem;

        text-align: center;

        color: #bbb;

        margin-top: 3rem;
    }

    .form-text {

        color: #aaa;
    }
</style>
</head>
<body>

    <!-- Navbar -->

    <nav class="navbar navbar-expand-lg navbar-dark bg-dark py-3">

        <div class="container">

            <a class="navbar-brand" href="/">🎮 GameZone</a>

            <button class="navbar-toggler" type="button" data-bs-
toggle="collapse" data-bs-target="#navMenu">

                <span class="navbar-toggler-icon"></span>

```

```

    </button>

    <div class="collapse navbar-collapse justify-content-end"
id="navMenu">

        <ul class="navbar-nav">

            <li class="nav-item"><a class="nav-link"
href="/">Home</a></li>

            <li class="nav-item"><a class="nav-link"
href="/about">About</a></li>

            <li class="nav-item"><a class="nav-link"
href="/contact">Contact</a></li>

            <li class="nav-item"><a class="nav-link"
href="/cart">Cart</a></li>

            <li class="nav-item"><a class="nav-link active text-info"
href="/register">Register</a></li>

            <li class="nav-item"><a class="nav-link"
href="/login">Login</a></li>

        </ul>

    </div>

</div>

</nav>

<!-- Registration Form -->

<main>

    <h1 class="animate__animated animate__fadeInDown">Create Your
Account</h1>

    <form onsubmit="validateRegister(event)">

        <div class="mb-3">

```



```
<label for="username" class="form-label">Username</label>

<input type="text" class="form-control" id="username" required
/>

</div>

<div class="mb-3">

  <label for="email" class="form-label">Email</label>

  <input type="email" class="form-control" id="email" required />

</div>

<div class="mb-3">

  <label for="phone" class="form-label">Phone Number</label>

  <input type="tel" class="form-control" id="phone" pattern="[0-9]{10}" placeholder="e.g. 9876543210" required />

</div>

<div class="mb-3">

  <label for="dob" class="form-label">Date of Birth</label>

  <input type="date" class="form-control" id="dob" required />

</div>

<div class="mb-3">

  <label for="genre" class="form-label">Favorite Game
Genre</label>

  <select class="form-control" id="genre" required>

    <option value="">Select one</option>
```

```
<option>Action</option>

<option>Adventure</option>

<option>RPG</option>

<option>Sports</option>

<option>Strategy</option>

<option>Simulation</option>

</select>

</div>

<div class="mb-3">

  <label for="password" class="form-label">Password</label>

  <input type="password" class="form-control" id="password"
minlength="6" required />

  <div class="form-text">Minimum 6 characters</div>

</div>

<div class="mb-3">

  <label for="confirmPassword" class="form-label">Confirm
Password</label>

  <input type="password" class="form-control"
id="confirmPassword" required />

</div>

<button type="submit" class="btn btn-primary mt-
3">Register</button>

</form>

</main>
```

```
<footer>

  <p>&copy; 2025 GameZone. All rights reserved.</p>

</footer>


<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"></script>


<script>

  function validateRegister(event) {

    event.preventDefault();

    const username =
document.getElementById("username").value.trim();

    const email = document.getElementById("email").value.trim();

    const phone = document.getElementById("phone").value.trim();

    const dob = document.getElementById("dob").value.trim();

    const genre = document.getElementById("genre").value.trim();

    const password =
document.getElementById("password").value.trim();

    const confirmPassword =
document.getElementById("confirmPassword").value.trim();

    if (!username || !email || !phone || !dob || !genre || !password
|| !confirmPassword) {

      alert("⚠ Please fill in all fields!");
```

```
        return;

    }

    if (password !== confirmPassword) {

        alert("✗ Passwords do not match!");

        return;

    }

    fetch("/register", {

        method: "POST",

        headers: { "Content-Type": "application/json" },

        body: JSON.stringify({ username, email, phone, dob, genre,
password })

    })

    .then(res => res.json())

    .then(data => {

        alert(data.message);

        if (data.message === "User registered successfully!") {

            window.location.href = "/login";

        }

    })

    .catch(err => {

        console.error(err);

        alert("Error connecting to server. Please try again.");

    });
```

```
}  
  
</script>  
  
</body>  
  
</html>
```

## Output-

 **GameZone**

HomeAboutContactCartRegisterLogin

### Create Your Account

Username

Email

Phone Number

Date of Birth

Favorite Game Genre

Password

Minimum 6 characters

Confirm Password

Register

© 2025 GameZone. All rights reserved.

The image shows a terminal window and a MySQL database interface. The terminal window displays the command to start the game-store application, which successfully starts the GameZone server at http://localhost:3000 and connects to the MySQL database. The MySQL interface shows the results of a query selecting all data from the 'users' table, displaying 5 rows of user information.

```
PS C:\Users\sujit\game-store> npm start

> game-store@1.0.0 start
> node server.js

✓ GameZone server running at http://localhost:3000
✓ Connected to MySQL database
```

SELECT \* FROM 'users'

☐ Profiling [ Edit inline ] [ Edit ] [ Explain SQL ] [ Create PHP code ] [ Refresh ]

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Extra options

|                                           | id | username | email                     | phone      | dob        | genre     | password   |
|-------------------------------------------|----|----------|---------------------------|------------|------------|-----------|------------|
| <input type="checkbox"/> Edit Copy Delete | 1  | sujit    | sumigood@gmail.com        | 9309115544 | 2006-09-19 | Adventure | Sujit8090  |
| <input type="checkbox"/> Edit Copy Delete | 2  | sujit    | sumigood@gmail.com        | 9309115544 | 2006-09-19 | Adventure | Sujit8090  |
| <input type="checkbox"/> Edit Copy Delete | 3  | sujit2   | sumigood@gmail.com        | 9309115544 | 2006-09-19 | Sports    | Sujit80901 |
| <input type="checkbox"/> Edit Copy Delete | 4  | sujit    | sumigood@gmail.com        | 9309113344 | 2025-11-28 | Strategy  | Sujit80901 |
| <input type="checkbox"/> Edit Copy Delete | 5  | sujit    | sujitbhagwat411@gmail.com | 9309115544 | 2025-11-10 | Adventure | Sujit80901 |

☐ Check all | With selected: Edit Copy Delete Export

☐ Show all | Number of rows: 25 | Filter rows: Search this table | Sort by key: None

Query results operations

Print Copy to clipboard Export Display chart Create view

Bookmark this SQL query

## Conclusion

RESTful APIs are the backbone of modern web services. Using Express.js, developers can build fast, secure, and scalable APIs to handle CRUD operations efficiently. They enable seamless data exchange across applications, forming the foundation of every full-stack ecosystem.